INSTITUT UNIVERSITAIRE DE TECHNOLOGIE DE LA ROCHELLE

Département Informatique

Stage BUT 3eme année

Teddy VALENTIN – BUT Informatique,

Parcours réalisation d'applications conception, développement, validation.

Stagiaire Développeur Full-Stack et DevOps IoT

Mise en place d'un environnement de micro-localisation de visiteurs en lieux fermés.

Période de stage : du *18/03/2024 au 26/06/2024*

Organisme d'accueil

Laboratoire Informatique, Image et Interaction (L3i), Av. Michel Crépeau, 17042 La Rochelle

Enseignant tuteur : *Mme. LASSUS Annick*

Maitre de stage : *Mr. FAUCHER Cyril*

<Consultable/confidentiel>

Année Universitaire 2023-2024

1 Glossaire 5
2 Remerciements 8
3 Introduction 9
4 Présentation du L3i 10
4.1 Description du Laboratoire de Recherche 10
4.2 Stratégie de Recherche 10
4.3 Domaines de contributions 11
4.4 Présentation de l'équipe et des différentes applications de la solution possible 12
5 Périmètre de la demande 14
6 Compétences acquises lors de la SAE à l'IUT de La Rochelle ... 15
7 Demandes et besoins 16
7.1 Matériels et outils mis à disposition 17
7.2 Objectif 1 : Détecter les visiteurs dans le musée 17
7.3 Objectif 2 : Collecter les données des badges 19
7.4 Objectif 3 : Orchestrer et automatiser les mises à jour des récepteurs 19
7.5 Objectif 4 : Héberger et traiter les données collectées 21
7.6 Objectif 5 : Assurer une connectivité Wi-Fi étendue et fiable 21
7.7 Objectif 6 : Suivre l'état des joueurs et visualiser les données 21
8 Recueillir et centraliser les données des badges BLE 22
8.1 Recueillir les données des badges BLE 22
8.2 Passer à une solution centralisée 23

8.3 Utilisation du filtre Grok de Logstash 27

9 Localiser les joueurs 29

9.1 Graphe des possibles 29

9.2 Algorithme : Graph Positioning 31

10 Configurer, gérer, orchestrer et visualiser l'environnement depuis un tableau de bord 36

10.1 Problèmes rencontrés avec le tableau de bord existant 36

10.2 Nouvelle solution proposée 37

10.3 Amélioration de la gestion de l'inventaire 38

10.4 Création d'une tâche permettant de récupérer automatiquement l'état des Raspberry 40

10.5 Nouvelles demandes concernant le tableau de bord 41

10.6 Conception des nouvelles fonctionnalités du tableau de bord 44

10.7 Conception d'une API 44

10.8 Conception d'un WebSocket pour Ansible 45

10.9 Détails des nouvelles fonctionnalités 49

11 Exemple d'intégration 54

11.1 Les Aventures de Rose 54

11.2 Le robot Pepper 55

12 Introspection et bilan personnel global 58

12.1 Un projet complexe par sa diversité 58

12.2 Méthodologie de travail 58

12.3 Compétences techniques acquises 59

12.4 Collaborations et interactions 59

13 Conclusion finale 60

14 Page de résumé en Français et Anglais 61

14.1 Résumé en Français : 61

14.2 Résumé en Anglais : 61

15 Annexe 63

15.1 Orchestration du déploiement de mise à jour grâce à Ansible sur un parc de Raspberry d'une taille n. 63

15.2 Détection des signaux BLE et lecture des données publicitaires 64

15.3 API pour la gestion de l'inventaire 67

15.4 API pour la gestion du graphe 68

15.5 API pour la gestion des joueurs 69

15.6 API pour la gestion des profils 70

15.7 Documentation des collections MongoDB 71

1 Glossaire

API (Application Programming Interface)	Interface de programmation permettant la communication entre différents logiciels. Dans le cadre du stage, l'API facilite les interactions entre le robot Pepper, la base de données MongoDB, et les divers composants du système de micro-localisation.
BLE (Bluetooth Low Energy)	Technologie de communication sans fil conçue pour des transferts de données à courte distance et optimisée pour une consommation d'énergie réduite. Utilisée pour la détection des badges des participants au jeu.
FileBeat	Logiciel léger pour le transfert de données de fichiers log vers un centre de stockage comme Logstash ou Elasticsearch. Il a été utilisé pour la remontée des données depuis les Raspberry Pi.
Graph Positioning	Algorithme permettant de déterminer la position d'un objet ou d'une personne dans un espace en se basant sur les signaux reçus par différents capteurs. Important pour la micro-localisation des visiteurs du musée.
IoT (Internet of Things)	Réseau d'objets physiques (« choses ») dotés de capteurs, de logiciels et d'autres technologies dans le but de se connecter et d'échanger des données avec d'autres dispositifs et systèmes sur Internet.

LogStash	Outil de traitement de logs permettant de collecter, transformer et transférer les données vers un stockage comme Elasticsearch. Utilisé pour traiter les données envoyées par FileBeat.
MongoDB	Base de données NoSQL orientée documents, conçue pour faciliter le développement et le scaling. Utilisée pour stocker les données de micro-localisation et d'interaction des joueurs.
Pepper (Robot)	Robot humanoïde programmable conçu pour interagir avec les humains. Dans le cadre du stage, Pepper est utilisé pour interagir avec les visiteurs du musée, posant des questions et guidant l'expérience de jeu.
Raspberry Pi	Ordinateur de la taille d'une carte de crédit utilisé pour divers projets d'électronique et d'informatique. Dans ce projet, plusieurs Raspberry Pi sont utilisés pour collecter les données de localisation des badges BLE.
Grok	Filtre puissant de Logstash permettant de parser et structurer les données de logs. Utilisé pour transformer les données des badges BLE en informations structurées avant leur stockage dans MongoDB.
Orbi Routeur/Satellite	Composants clés du système Orbi, où le routeur se connecte à l'internet et les satellites se placent stratégiquement autour de l'espace pour étendre la couverture Wi-Fi.

Ansible	Outil d'automatisation IT permettant de gérer et configurer des serveurs de manière automatisée
NUC (Next Unit of Computing)	Petit ordinateur utilisé comme serveur central pour héberger les algorithmes et outils nécessaires au projet. Il centralise la gestion de l'infrastructure et stocke les données collectées.
Swagger	Framework open source utilisé pour la conception, la construction, la documentation et la consommation de services Web RESTful. Utilisé dans le projet pour documenter et tester l'API développée.

2 Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes et institutions qui ont rendu ce projet possible.

Tout d'abord, un grand merci à M. Cyril Faucher, mon maître de stage, pour son accompagnement, sa patience et son expertise. Votre guidance m'a permis de développer de nouvelles compétences et d'avancer sereinement dans ce projet complexe.

Merci à l'équipe du Laboratoire Informatique, Image et Interaction (L3i), notamment M. Damien Mondou et M. Jérémy Richard, pour leur collaboration, leur assistance technique et leur esprit d'équipe. Votre aide a été essentielle à la réalisation de ce projet.

Enfin, je tiens à remercier l'Institut Universitaire de Technologie de La Rochelle et son Département Informatique pour cette opportunité de stage, ainsi que le Muséum d'Histoire Naturelle de La Rochelle pour avoir accueilli notre expérimentation.

Je remercie également Mme Annick Lassus pour son soutien tout au long de cette expérience.

Grâce à l'implication et au soutien de chacun, ce projet a pu se concrétiser et m'a permis de vivre une expérience enrichissante et formatrice.

3 Introduction

Dans le cadre de ma troisième année du Bachelor Universitaire de Technologie en Informatique, spécialité réalisation d'applications : conception, développement, validation, j'ai eu l'opportunité de réaliser mon stage au sein du Laboratoire Informatique, Image et Interaction (L3i) de La Rochelle Université, entre le 18 mars et le 26 juin 2024. Cette expérience m'a permis de me plonger au cœur d'un projet innovant visant à enrichir l'expérience culturelle des visiteurs du Muséum d'histoire naturelle de La Rochelle, à travers le développement d'une solution de micro-localisation en temps quasi réel.

Sous la tutelle de M. Cyril Faucher, j'ai contribué à la mise en place d'un environnement de micro-localisation de visiteurs en lieux fermés. Ce projet est lié au consortium « J'peux pas, j'ai musée », qui développe des solutions numériques visant à intégrer davantage la culture dans le monde numérique moderne, afin de reconnecter la culture dans un monde hyper centré sur l'information et le numérique.

4 Présentation du L3i

4.1 Description du Laboratoire de Recherche

Le Laboratoire Informatique, Image et Interaction (L3i) de La Rochelle Université créé en 1993, s'associe aux chercheurs en informatique de l'IUT et de la Faculté des Sciences et Technologies de La Rochelle Université. Avec près de 100 membres, le laboratoire est lié au monde de la recherche en informatique, consolidant une politique de réseau importante et cultivant une relation forte avec le monde socio-économique, notamment à travers un consortium académico-industriel.

4.2 Stratégie de Recherche

La stratégie de recherche du L3i repose sur la gestion intelligente et interactive des contenus numériques, avec un focus particulier sur les contenus créés par et pour l'homme. Cette orientation stratégique se traduit par des projets de recherche ambitieux adressant des problématiques jugées stratégiques sur le plan européen et national. Le laboratoire développe ainsi un projet de recherche résolument tourné vers les enjeux de société et s'articule autour de trois grandes thématiques scientifiques : les modèles et connaissances, les images et contenus, ainsi que la dynamique des systèmes et l'adaptativité. Ces axes permettent au L3i d'aborder de manière holistique la valorisation des contenus numériques.

4.2.1 Présentation équipe axes

L'équipe "Modèles et connaissances" est constituée de 12 enseignants chercheurs permanents, 9 doctorants et 5 post-doctorants. Elle se divise en deux thèmes de recherche : "Modèles Formels" et "Connaissances". Les travaux portent sur les modèles formels, les données spatio-temporelles, la modélisation de connaissances et la gestion des données variées.

L'équipe "Images et contenus" regroupe des chercheurs spécialisés dans le traitement bas niveau des contenus faiblement structurés. Les objectifs incluent l'analyse, la gestion, l'extraction de caractéristiques, et la structuration des espaces de caractéristiques pour un accès rapide et efficient.

L'équipe "Dynamique des systèmes et adaptativité" travaille sur la dynamique des systèmes logiciels et l'adaptativité, incluant les interactions homme-système et homme-contenu. Les recherches visent à gérer l'explosion massive des données hétérogènes et à proposer des environnements numériques adaptés aux utilisateurs.

4.3 Domaines de contributions

Depuis sa création en 1993, le L3i s'est concentré sur une thématique liée à la gestion intelligente et interactive des contenus numériques, s'attaquant à des problématiques cruciales pour la société moderne. Les contributions et projets phares du L3i sont divers et couvrent plusieurs axes de recherche, notamment :

1. **Traitement et analyse d'images :** Les recherches dans ce domaine incluent le traitement des contenus faiblement structurés (images, textes, vidéos, documents numériques) et l'analyse, la gestion, et la mise en relation des données extraites de ces contenus. Ces travaux visent à traiter les informations bas niveaux, extraire des signatures pertinentes, structurer les espaces de caractéristiques pour un accès rapide et efficient, et projeter les contenus dans des espaces de caractéristiques adaptés pour favoriser les processus interactifs d'indexation, de navigation, de recherche d'informations.
2. **Interaction homme-machine (IHM) :** L'axe IHM du L3i explore la dynamique des systèmes logiciels et l'adaptativité de ces systèmes, incluant la dynamique des systèmes en réseaux et l'autoadaptation. Les recherches portent sur le pilotage auto-adaptatif des infrastructures et des services numériques du futur, vers des infrastructures numériques intelligentes, éco-efficientes, et sécurisées ainsi que des approches centrées sur l'humain.

3. **Intelligence Artificielle (IA) :** Au cœur de ses recherches sur l'IA, le L3i développe des modèles et connaissances, incluant l'étude des modèles formels et les données spatio-temporelles, l'intégration de données par les modèles et les raisonnements associés, la fouille de données à travers de réseaux complexes et l'analyse formelle des concepts. Ces recherches contribuent à l'analyse de données complexes en développant et en utilisant des outils théoriques et en les appliquant dans le contexte des données numériques produites par et pour l'homme.

Le L3i se distingue non seulement par sa contribution académique mais aussi par son engagement fort avec le monde socio-économique, développant des projets de recherches tournés vers les enjeux de société et adressant des problématiques stratégiques. La politique scientifique du laboratoire est ancrée sur des enjeux socio-économiques, affirmant son rôle comme un acteur clé dans la recherche et le développement de technologies innovantes pour répondre aux défis contemporains.

Régionalement, il fait partie de la Fédération CNRS MIRES et de l'ERT « Interactivité numérique », lui permettant de collaborer étroitement avec des acteurs locaux. Au niveau national, sa participation aux groupes de recherche GDR I3 et ISIS souligne son engagement envers les initiatives scientifiques françaises. Sur le plan international, l'affiliation avec l'IAPR et les partenariats avec des centres de recherche de premier plan à l'échelle mondiale (Espagne, Japon, Vietnam, Tunisie, Malaisie) témoignent de son influence et de son ouverture globale.

4.4 Présentation de l'équipe et des différentes applications de la solution possible

Mon projet étant intégré dans l'équipe Modèle et Connaissances j'ai pu interagir avec plusieurs doctorants tel : M. Damien Mondou et M. Jérémy Richard, m'ayant assisté dans mon projet au côté de mon maître de stage M. Cyril Faucher.

L'objectif principal de mon stage est alors de développer une suite de solutions technologiques pour faciliter la localisation précise des joueurs dans un environnement en lieux fermés, permettant ainsi l'intégration de divers Serious Games. Pour ce faire, j'ai mis en place un système de collecte de données via des Raspberry Pi et FileBeat, orchestré ce système avec Ansible, géré les données collectées dans une base MongoDB, et développé une API pour le suivi de l'état du jeu. De plus, j'ai travaillé sur l'amélioration de l'algorithme de Graph Positioning pour une localisation précise et le suivi des joueurs.

5 Périmètre de la demande

L'un des projets du L3i est d'intégrer le numérique à la culture afin de proposer des expériences alternatives aux services culturels. L'objectif est de permettre aux différents types d'utilisateurs de vivre des expériences uniques grâce à la micro-localisation en quasi-temps réel. Cela inclut la collecte de données et la visualisation des parcours des visiteurs au sein du musée, ainsi que la proposition d'outils permettant d'intégrer cet environnement à d'autres applications et de permettre l'intégration de ces données de micro-localisation.

Dans le cadre de mon stage, je dois être en mesure d'intégrer cet environnement au Muséum d'Histoire Naturelle de La Rochelle comme lieu d'expérimentation afin de démontrer que l'environnement est fonctionnel et permet de répondre aux besoins, tout en fournissant un cadre suffisamment modulable pour pouvoir intégrer ce système dans plusieurs applications culturelles à l'avenir. Le musée n'est pas le demandeur du projet, mais sert de site pilote pour l'expérimentation et la validation de notre solution technologique.

Par la même occasion, l'environnement doit être configurable et pouvoir être adapté à des scénarii différents. Par exemple, en fonction du lieu, on veut pouvoir mettre en place et configurer l'environnement en fonction de la disposition et des caractéristiques de l'environnement.

Cela demande alors une certaine modularité exigeant une certaine rigueur afin que l'environnement s'adapte avec la plus grande simplicité possible.

6 Compétences acquises lors de la SAE à l'IUT de La Rochelle

Ce projet de stage est en lien avec une Situation d'Apprentissage et d'Évaluation (SAE) à l'IUT de La Rochelle. En effet, il a été initié lors d'une SAE destinée à confier des projets aux élèves, en leur donnant un objectif et une mission pour les sensibiliser et les stimuler afin de réaliser des projets avec de véritables enjeux, tout en améliorant leurs compétences. Durant ce projet, j'ai eu l'opportunité de travailler avec M. Cyril Bonnard, M. Alexandre Barreau et M. Nils Galloux.

Lors de la SAE, j'ai eu un aperçu de ce que serait le stage. Bien que la SAE et le stage n'abordent pas exactement les mêmes sujets, ils restent néanmoins liés par les compétences techniques acquises. En effet, lors de cette SAE, notre objectif était d'automatiser le déploiement de code sur une infrastructure de 20 Raspberry Pi. Cette expérience m'a permis de me familiariser avec certaines technologies que j'ai ensuite utilisées pendant mon stage, telles qu'Ansible, Raspberry Pi Zéro W et BLE. Notre mission consistait également à récupérer les trames des signaux BLE et à les enregistrer dans un fichier. Bien que rudimentaire comparé à l'environnement développé par la suite, cela m'a également permis de me familiariser avec une forme de remontée de données.

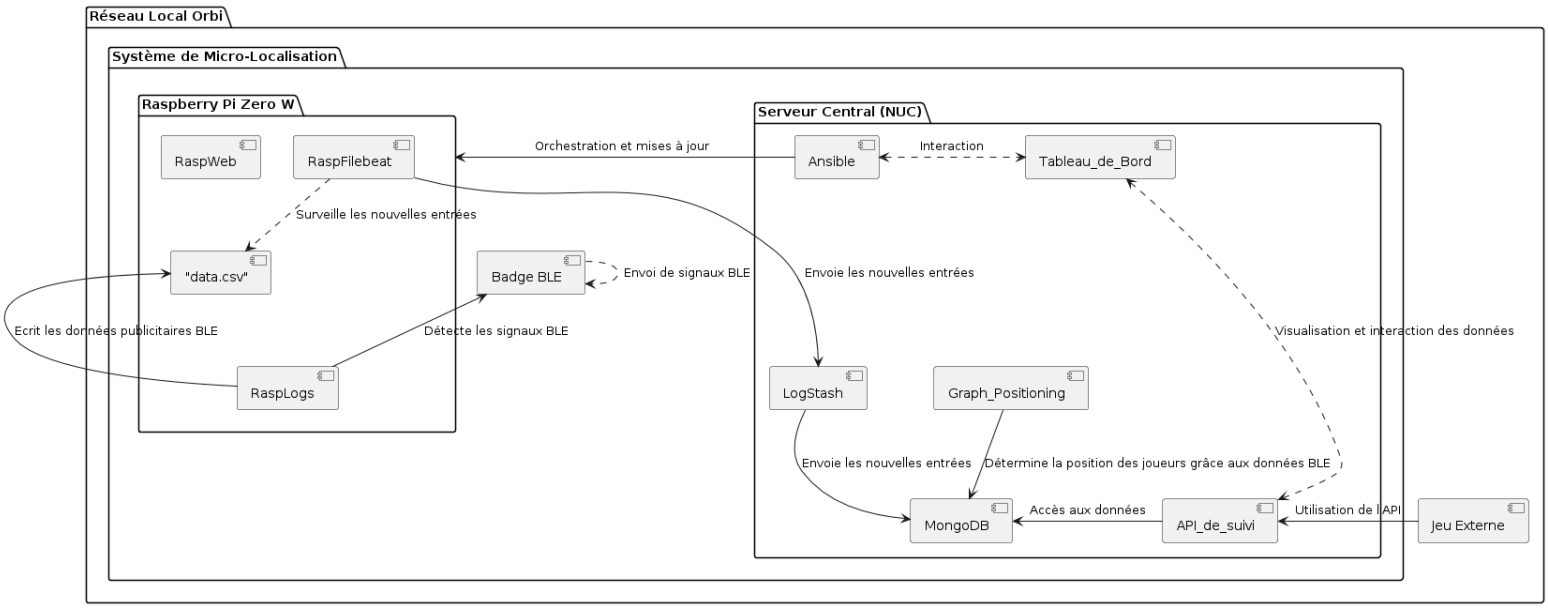
En outre, cette SAE a été très formatrice, me permettant de découvrir de nombreuses technologies que je retrouve dans mon stage et de mieux comprendre avec quoi je travaille aujourd'hui. Elle a offert une base solide pour construire un système plus complexe et m'a permis de monter en compétence sur des briques essentielles de l'infrastructure actuelle.

7 Demandes et besoins

La demande pour le stage était alors la suivante : « Créer un environnement de micro-localisation de visiteurs en quasi-temps réel en lieux fermés. ». On peut alors déconstruire cette demande en quelques grands axes :

- Recueillir et centraliser les données des badges BLE
- Localiser les joueurs grâce à un algorithme de positionnement
- Visualiser les données depuis un tableau de bord
- Avoir la capacité de configurer et gérer les sessions de collecte depuis un tableau de bord

Chaque aspect de cette demande requiert une infrastructure technologique spécifique, pour laquelle j'ai pu acquérir des compétences préalables lors de la SAE. Ainsi, nous pouvons établir une architecture globale dont chaque partie sera définie progressivement tout au long du rapport.



7.1 Matériels et outils mis à disposition

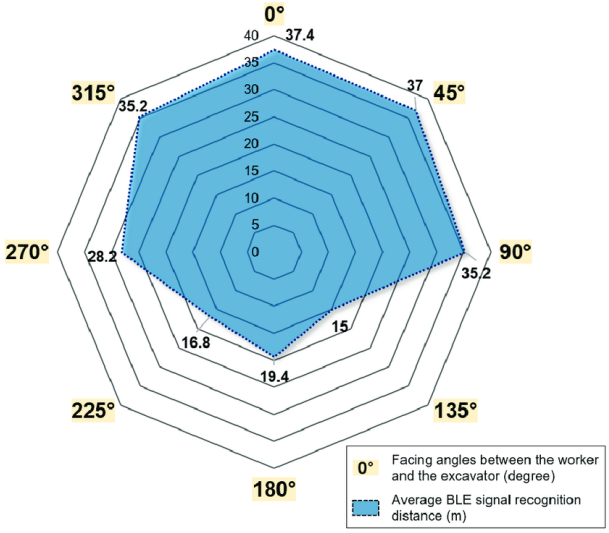
Afin de mettre en place cette environnement des matériels et outils m'ont alors été mis à disposition afin de réaliser cet environnement.

7.2 Objectif 1 : Détecter les visiteurs dans le musée

7.2.1 Solution: Badge iBeacon BLE (Bluetooth Low Energy)

Chaque visiteur du muséum est équipé d'un badge unique permettant de les identifier dans le musée par l'environnement. Chaque badge est composé de deux composantes

Une petite pile permettant la transmission du signal BLE (Bluetooth Low Energy) dans ses alentours. Ce système BLE est fonctionnellement similaire au Bluetooth mais ne permet pas une connexion stable entre deux appareils ; il émet seulement à intervalles réguliers des signaux que n'importe quel appareil équipé de la technologie nécessaire peut alors identifier et traiter l'information transmise. La technologie iBeacon, développée par Apple, permet de composer des systèmes de positionnement en intérieur. Cependant, elle est limitée par la physique des signaux, étant affectée par les matériaux



d'un bâtiment, son isolation et l'orientation de l'émetteur et du récepteur.

Un signal iBeacon se décompose en plusieurs composantes qui permettent la localisation :

- UUID : Similaire à une adresse MAC permettant l'identification unique du transmetteur.
- Major/Minor : Valeurs permettant de filtrer les signaux de manière simple. Dans notre cas, le major permet de savoir à quelle fréquence nos badges BLE émettent et le minor permet d'identifier chaque badge indépendamment.
- RSSI (Indicateur de force du signal reçu) : Mesure représentant le niveau de qualité relative d'un signal Bluetooth reçu sur un appareil.
- TX Power : Mesure de RSSI pour laquelle le transmetteur et le récepteur se trouvent à 1m de distance.



***Figure 1**: When a device is further away from the beacon, signal strength is diminished and therefore the accuracy estimate will increase.*

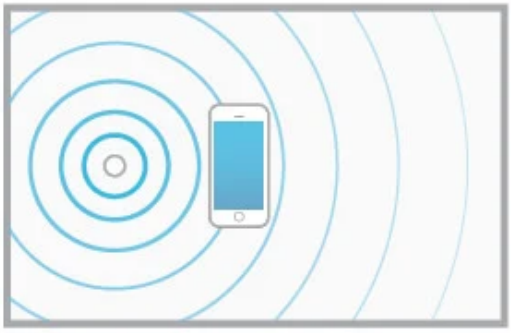


***Figure 2**: Signal strength increases as a device moves closer to the beacon, leading to a better proximity estimate.*

Cette technologie est essentielle dans notre projet, nous permettant de mettre en place une localisation de signaux et de pouvoir identifier approximativement à quelle distance se trouvent un émetteur et un récepteur. Elle est également très robuste et a une faible consommation, permettant une très grande longévité et nécessitant très rarement une recharge.

7.3 Objectif 2 : Collecter les données des badges

7.3.1 Solution : Raspberry Pi Zéro W

Afin de pouvoir recevoir les signaux BLE transmis par les badges des visiteurs, nous utilisons des micro-ordinateurs capables d'analyser, filtrer et transmettre les données vers un serveur de collecte capable ensuite de sauvegarder les différentes valeurs. Les Raspberry Pi Zero W sont choisis pour leur faible consommation en énergie et leur capacité à détecter et traiter les données de manière efficace. Cependant, ces Raspberry ont des limites matérielles qu'il est nécessaire de respecter :

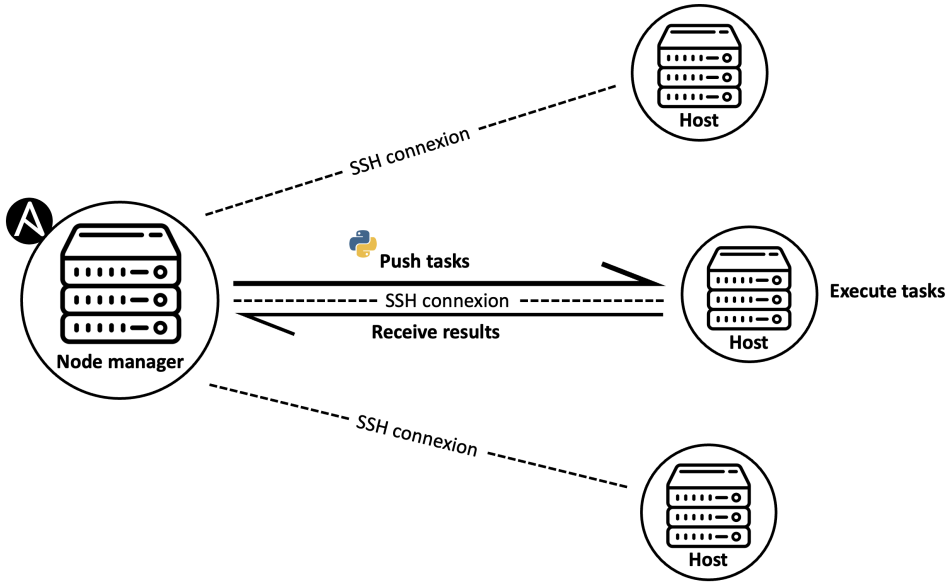
- Architecture : ARMV6l (32-bit)
- Mémoire RAM : 512MB LPDDR2
- Processeur : 1Ghz ARM11 monocoeur

Ces limitations doivent être prises en compte pour ne pas surcharger le Raspberry de tâches qu'il ne serait pas capable de supporter. Il est donc nécessaire d'implémenter des outils peu lourds et efficaces afin de limiter l'utilisation des performances inutiles.

7.4 Objectif 3 : Orchestrer et automatiser les mises à jour des récepteurs

7.4.1 Solution : Ansible

Ansible, installé sur le serveur central (NUC), permet de gérer et d'orchestrer les mises à jour des Raspberry Pi. Il automatise les déploiements de manière presque autonome et effectue un reporting sur les actions ayant échoué ou réussi. Cette orchestration est essentielle pour maintenir le système à jour sans intervention manuelle sur chaque Raspberry Pi. De plus cet outil permet de lancer les différents services essentiels aux Raspberry Pi, permettant de lancer une session de collecte ou de l'arrêter.



Ansible exécute alors des « playbooks [1]», permettant d'exécuter des tâches dans un ordre chronologique tel que :

```yaml
---
- name: Variables playbook
  hosts: all
  vars:
      state: latest
      user: bob
  tasks:
  - name: Add the user {{ user }}
    ansible.builtin.user:
      name: "{{ user }}"
  - name: Upgrade all apt packages
    apt:
      force_apt_get: yes
      upgrade: dist
  - name: Install the {{ state }} of package "nginx"
    apt:
      name: "nginx"
      state: "{{ state }}"
```

Dans cet exemple, on exécute les tâches suivantes sur tous les hôtes distants tel que :

- On créer un utilisateur nommé « bob »
- On met à jour tous les packages en utilisant apt
- Et on installe la dernière version de nginx.

Toutes ces tâches sont alors définite dans un yml permettant d'écrire des tâches plutôt simplement.

[1] https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_intro.html

7.5 Objectif 4 : Héberger et traiter les données collectées

7.5.1 Solution : Serveur central (NUC)

Un ordinateur de petite taille, le NUC, permet d'héberger les algorithmes les plus lourds et les divers outils nécessaires pour le Serious Game. Ce serveur centralise la gestion de l'infrastructure, héberge la base de données MongoDB pour stocker les données collectées, et exécute l'algorithme de Graph Positioning qui utilise les données BLE pour localiser les joueurs et mettre à jour leurs informations dans MongoDB.

7.6 Objectif 5 : Assurer une connectivité Wi-Fi étendue et fiable

7.6.1 Solution : Orbi Router et Satellite

Le système Orbi est utilisé pour assurer une couverture Wi-Fi étendue et fiable dans tout l'espace du musée. Ce système de réseau maillé (mesh network) est composé d'un routeur principal (Orbi Router) et de plusieurs satellites. Il permet de créer un réseau unique couvrant l'intégralité des zones d'exposition sans zones mortes ni déperdition de signal, assurant ainsi une connectivité constante et efficace pour les dispositifs IoT et les Raspberry Pi impliqués dans la collecte des données de localisation des visiteurs.

7.7 Objectif 6 : Suivre l'état des joueurs et visualiser les données

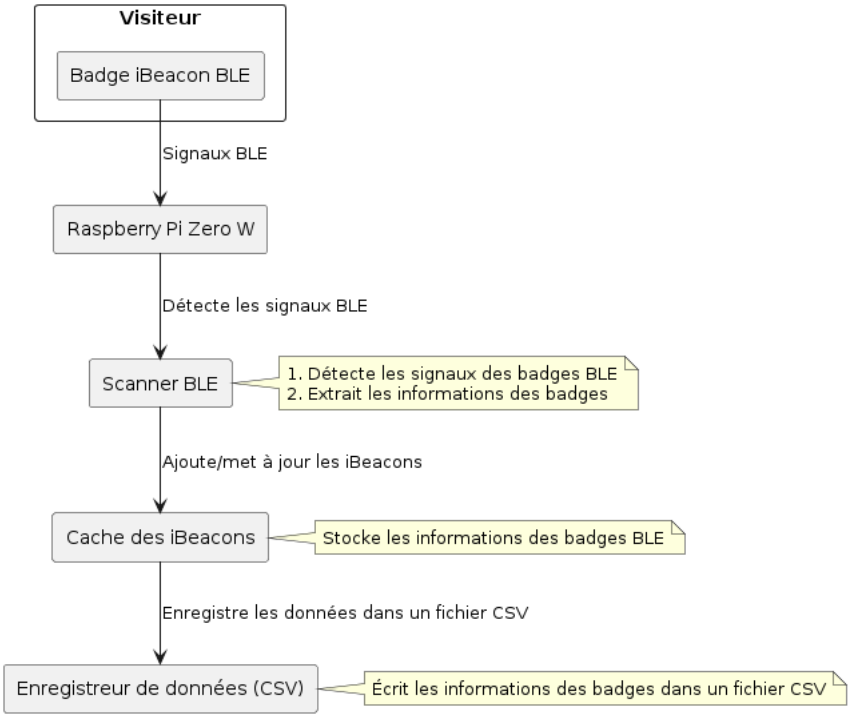
7.7.1 Solution : API de suivi et Tableau de bord

L'API, également hébergée sur le NUC, permet de récupérer les informations des joueurs depuis une source de jeu externe, de créer des joueurs, et de transmettre des informations à un tableau de bord pour la visualisation. Cette API facilite l'interaction en temps réel et la gestion des données, offrant une vue d'ensemble du système et de l'état des joueurs.

8 Recueillir et centraliser les données des badges BLE

8.1 Recueillir les données des badges BLE

Afin de recueillir et centraliser les données des badges BLE pour ce faire il faut déjà pouvoir détecter les badges BLE l'annexe page 64 décrit ce procédé ayant été réalisé pendant la SAE par moi-même je préfère donc le mettre en annexe. Ci-dessous on peut également définir ce procédé de récolte des données telles que :



8.1.1 Format de sauvegarde du fichier data.csv

Le fichier .csv contient les données des iBeacons détectés. Chaque ligne correspond à un signal iBeacon BLE détecté. Les données sont séparées par des virgules.

Le format de chaque ligne est le suivant : identity,timestamp,major,minor,tx_power,rssi

Variable	Description
identity	Identité de l'utilisateur ayant exécuté le script de détection du badge BLE. Les Raspberry sont configurés avec un nom et un numéro tel qu'une identité : « rasp11 » par exemple.
timestamp	Timestamp de l'instant t de la détection du signal du badge BLE.
major	Similaire à un numéro de fréquence permet de filtrer les signaux. Chaque badge à le même major.
minor	Similaire à un numéro de fréquence permet de filtrer les signaux. Chaque badge à un minor différent.
tx_power	Valeur RSSI pour un distance égale à un mètre entre le récepteur et l'émetteur.
rssi	Valeur de la force du signal.

8.2 Passer à une solution centralisée

Après avoir recueilli toutes ces données localement, il devient crucial de les envoyer à un emplacement centralisé pour les traiter et les analyser efficacement en quasi-temps réel. Le stockage local des données dans des fichiers CSV est une solution temporaire et limitée ne permettant pas une analyse avec toutes les données des Raspberry Pi agglomérées où alors serait très lente. Pour une gestion plus robuste et scalable des données, une infrastructure centralisée est nécessaire.

8.2.1 Contexte et choix de FileBeat et Logstash

« Filebeat is a lightweight shipper for forwarding and centralizing log data. Installed as an agent on your servers, Filebeat monitors the log files or locations that you specify, collects log events, and forwards them either to Elasticsearch or Logstash for indexing. »[2]

Initialement, nous avons exploré la stack ELK[3] (ElasticSearch, LogStash, Beats et Kibana) pour cette tâche (ce qui était par ailleurs utilisé par le passé). La stack ELK est une solution populaire pour la collecte, le traitement et la visualisation des

[2] https://www.elastic.co/guide/en/beats/filebeat/current/filebeat-overview.html#:~:text=Filebeat%20is%20a%20lightweight%20shipper,Elasticsearch%20or%20Logstash%20for%20indexing

[3] https://www.elastic.co/fr/elastic-stack

données de logs. Cependant, nous avons constaté que l'installation de l'ensemble de la stack ELK pouvait être lourde et complexe pour notre cas d'utilisation.

8.2.2 Pourquoi ne pas utiliser toute la stack ELK ?

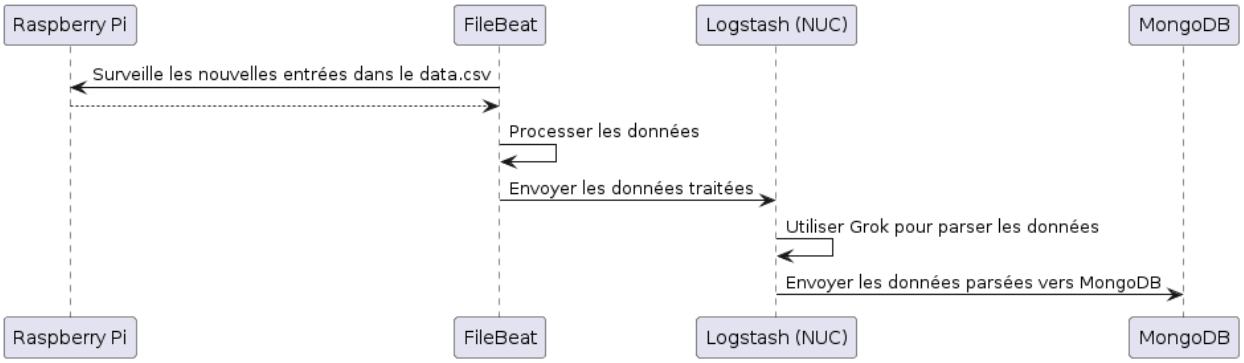
- Complexité : Installer et maintenir ElasticSearch, LogStash et Kibana nécessite des ressources significatives en termes de configuration et de gestion, également ce sont des applications lourdes en ressource.
- Surdimensionné : Pour notre projet, nous n'avons pas besoin de toutes les fonctionnalités offertes par Kibana ni par ElasticSearch pour la visualisation. Notre principal besoin est de centraliser et traiter les données.

8.2.3 Pourquoi FileBeat et Logstash ?

Pour répondre à nos besoins spécifiques de manière plus légère et efficace, nous avons choisi d'utiliser uniquement FileBeat et Logstash, qui offrent les avantages suivants :

- Légèreté et simplicité : FileBeat est un agent léger qui s'installe facilement sur les Raspberry Pi. Il permet de surveiller les fichiers logs et d'expédier les données vers Logstash sans nécessiter une infrastructure complexe.
- Modularité : En utilisant FileBeat, nous pouvons surveiller les fichiers logs sur chaque Raspberry Pi et envoyer ces logs à Logstash pour traitement.
- Flexibilité : Logstash, avec ses puissants filtres (notamment le filtre Grok[4]), permet de parser et structurer les données de manière flexible avant de les envoyer vers notre base de données MongoDB.

[4] https://www.elastic.co/guide/en/logstash/current/plugins-filters-grok.html



8.2.4 Utilisation de FileBeat

FileBeat est configuré pour surveiller les fichiers logs sur les Raspberry Pi et envoyer les données à Logstash. Voici un exemple de configuration de FileBeat :

```
---
filebeat.inputs:
  - type: log
    enabled: true
    paths:
      - /home/admin/data/*.csv

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
  reload.enabled: false

setup.template.settings:
  index.number_of_shards: 1

setup.kibana: null

output.logstash:
  hosts:
    - 192.168.2.1:5044

processors:
  - add_host_metadata:
      when.not.contains.tags: forwarded
  - add_cloud_metadata: null
  - add_docker_metadata: null
  - add_kubernetes_metadata: null
  - drop_fields:
      fields: ["host", "input", "agent"]
      ignore_missing: true

```

8.2.4.1 Explication de la configuration

- filebeat.inputs : Définit les chemins des fichiers logs que FileBeat doit surveiller. Dans notre cas, il s'agit de tous les fichiers CSV dans le répertoire /home/admin/data/.
- filebeat.config.modules : Spécifie le chemin des modules de configuration de FileBeat.
- setup.template.settings : Configure les paramètres de l'index.
- setup.kibana : La configuration de Kibana est laissée vide car nous n'utilisons pas Kibana.
- output.logstash : Définit l'adresse du serveur Logstash où FileBeat doit envoyer les données.
- processors : Ajoute des métadonnées et supprime certains champs des logs.

8.2.5 Défis d'installation de FileBeat sur Raspberry Pi

Installer FileBeat sur les Raspberry Pi n'a pas été simple, car les fichiers d'installation disponibles ne sont pas toujours compatibles avec l'architecture du Raspberry Pi. Il a donc été nécessaire de recompiler FileBeat pour l'adapter à notre architecture. Pour ce faire on peut utiliser des outils développés par la communauté tel que : https://github.com/vrince/arm-beats permettant de recompiler FileBeat à notre souhait en utilisant Go.

8.2.5.1 Utilisation de Go pour recompiler FileBeat

Pour recompiler FileBeat, le langage de programmation Go a été utilisé. Développé par Google, Go est particulièrement adapté pour cette tâche pour plusieurs raisons :

- Portabilité : Go permet de compiler des programmes pour différentes architectures (comme ARM, utilisée par les Raspberry Pi) à partir d'une même base de code.
- Performance : Go est conçu pour être performant, avec des temps de compilation rapides et des exécutions efficaces.

- Simplicité : Le langage Go est simple à utiliser, avec une syntaxe claire et des outils de gestion des dépendances intégrés.

Voici les étapes principales suivies pour recompiler FileBeat avec Go :

- Cloner le dépôt de code source de FileBeat : Le code source de FileBeat a été téléchargé depuis le dépôt officiel.
- Configurer l'environnement Go : L'environnement de développement a été configuré pour utiliser Go, en définissant les chemins nécessaires.
- Compiler le code pour ARM : Go a été utilisé pour compiler le code source de FileBeat en spécifiant l'architecture ARM.
- Créer une archive : Une fois la compilation terminée, une archive contenant le binaire compilé de FileBeat a été créée.

Grâce à ces étapes, une version de FileBeat compatible avec l'architecture ARM a été obtenue, adaptée pour une installation sur les Raspberry Pi que nous utilisons dans notre environnement.

8.3 Utilisation du filtre Grok de Logstash

Le filtre Grok de Logstash est particulièrement utile pour parser et structurer les données issues des badges BLE. Les types de données collectées comprennent toutes les données du data.csv qu'on va simplement envoyer et que logstash va pouvoir traiter et indexer dans une base de données.

```
"%{WORD:hostname},%{NUMBER:timestamp:float},%{NUMBER:major:int},%{NUMBER:minor:int},%{NUMBER:tx_power:int},%{NUMBER:rssi:int}"
```

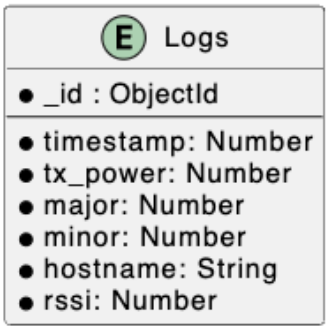
Le filtre Grok permet de parser ces données en les structurant sous forme de champs compréhensibles et utilisables. Une fois les données structurées, Logstash peut les envoyer directement vers une base de données MongoDB. Ce choix offre une modularité accrue par rapport à l'utilisation de toute la stack ELK. MongoDB est une base de

données légère, flexible et facile à utiliser, ce qui la rend idéale pour notre besoin de centralisation et d'analyse des données. Logstash permet également de configurer l'ensemble du processus de collecte et de traitement des données via un simple fichier `.conf`, ce qui est assez simple à comprendre et fonctionne de manière dynamique.

8.3.1 Défis et solutions

Cependant, de base, Logstash ne supporte pas l'envoi des données vers MongoDB. Il est donc nécessaire d'installer des modules complémentaires pour permettre cela.

8.3.2 Stockage des données



Chaque Raspberry embarque alors un FileBeat étant en capacité d'envoyer des données à LogStash.

Et donc grâce aux données remontées on peut définir une structure de données que l'on stocke dans une collection MongoDB permettant leur analyse ultérieure.

9 Localiser les joueurs

9.1 Graphe des possibles

Pour localiser les joueurs dans un espace en intérieur, il est crucial de positionner stratégiquement les Raspberry Pi et de connaître leur position précise. Chaque Raspberry Pi reçoit les signaux BLE émis par les badges des visiteurs, et l'algorithme détermine le Raspberry Pi le plus approprié (le meilleur candidat) pour chaque signal détecté. Voici comment ce processus est réalisé :

- Couverture Complète : Assurer une couverture complète de l'espace en plaçant les Raspberry Pi à des intervalles réguliers pour capter les signaux BLE de manière continue.
- Zones Clés : Placer les Raspberry Pi dans des zones clés comme les entrées, les sorties, les intersections de couloirs et les principales zones d'exposition.
- Minimiser les Interférences : Positionner les Raspberry Pi de manière à minimiser les interférences dues aux matériaux de construction, aux meubles ou à d'autres obstacles.

Chaque Raspberry Pi doit connaître sa position exacte dans l'espace pour permettre une localisation précise des badges BLE. Cette position est enregistrée dans une base de données centrale ou dans les configurations de chaque Raspberry Pi. La précision de la localisation dépend directement de la connaissance des positions des Raspberry Pi et de leur capacité à détecter les signaux BLE avec précision.

Définition d'un Graphe des Possibles

Un graphe des possibles est une représentation graphique qui modélise les différents chemins ou transitions possibles entre des états ou des positions dans un environnement donné. Dans le contexte de la localisation en intérieur, un graphe des possibles est utilisé

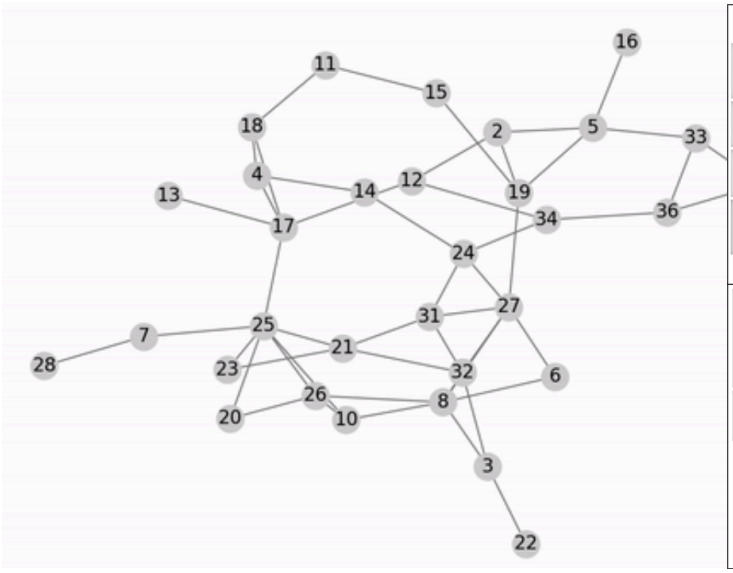
pour représenter les différents chemins que peuvent emprunter les visiteurs à travers un espace équipé de capteurs ou de balises.

Les Raspberry Pi sont liés entre eux par un graphe des possibles, qui simule les différents déplacements possibles entre deux Raspberry Pi. Ce graphe est utilisé pour :

- Modéliser les Chemins : Définir les chemins potentiels que les visiteurs peuvent emprunter à travers le musée. Chaque noeud du graphe représente un Raspberry Pi, et chaque arête représente un chemin possible entre deux Raspberry Pi.
- Calculer les Distances : Utiliser les mesures RSSI (Indicateur de force du signal reçu) pour estimer les distances entre les badges BLE et les Raspberry Pi, puis déduire la position des visiteurs.
- Suivre en Temps Réel : Mettre à jour en temps réel les positions des visiteurs en suivant le signal BLE capté par les différents Raspberry Pi.
-

Exemple du graphe des possibles des Raspberry placé à travers le musée :

Sur ce graphe on considère alors que chaque nœud est un Raspberry et le graphe étant la représentation mathématique du placement des Raspberry et du chemin entre les différents Rapsberry à travers le musée





9.2 Algorithme : Graph Positioning

Lorsqu'un badge BLE émet un signal, plusieurs Raspberry Pi peuvent le capter avec différentes intensités de signal (RSSI). Il faut alors pouvoir filtrer et calculer le meilleur candidat dans un groupe de données brut. On considère alors le Raspberry Pi avec le meilleur score calculé et ses poids tel que :

Paramètre	Valeur	Description
average_rssi_weight	0.7	Poids appliqué à la force du signal RSSI (Received Signal Strength Indicator).
beacon_count_weight	0.3	Poids appliqué à la quantité de fois qu'un signal d'un badge BLE est détecté.
distance_weight	0.2	Poids appliqué à la distance parcourue par le joueur (ou badge) depuis le dernier point visité.
distance_min	0	Distance minimale considérée pour la pondération par le poids de distance.
distance_max	4	Distance maximale considérée pour la pondération par le poids de distance.
rssi_min	-94	Valeur RSSI minimale utilisée pour la pondération par le poids de la force du signal.
rssi_max	-56	Valeur RSSI maximale utilisée pour la pondération par le poids de la force du signal.
confidence_threshold	0.3	Seuil de confiance minimal pour considérer un trajet comme valide.
time_interval_of_groups	8 secondes	Intervalle de temps en minutes pour regrouper les entrées de log (très important afin de pouvoir décomposer la trace en petit mouvements)

Groupement des données basé sur une fenêtre temporelle

Soit T l'ensemble des timestamps des signaux BLE détectés et Δt la fenêtre temporelle en secondes. Nous voulons regrouper les données en fonction de cette fenêtre temporelle.

Pour chaque signal i avec timestamp t_i :

$$\text{grouped_data} = \{\text{groupe}_k \mid \forall i, t_i \in T \land (t_i - t_{i-1} \leq \Delta t)\}$$

$$\text{last_data_time} = \begin{cases} \text{grouped_data}[-1][-1][\text{DATE_KEY}] & \text{si grouped_data existe} \\ \text{None} & \text{sinon} \end{cases}$$

Validation des groupes de données

Un groupe de données est valide si le nombre d'éléments dans le groupe est supérieur à 10 :

$$\text{validate_group_data}(G) = |G| > 10$$

Sélection du meilleur candidat

Pour chaque groupe de données G, nous calculons les fréquences des balises, la moyenne des RSSI, et les distances par rapport au dernier point visité.

Fréquence des balises

Pour chaque balise k dans le groupe G :

$$\text{beacon_count}[k] = \frac{\text{count}(k) - \min(\text{beacon_count})}{\max(\text{beacon_count}) - \min(\text{beacon_count}) + \epsilon}$$

Lissage et normalisation des RSSI

Pour chaque balise k dans le groupe G :

$$\text{rssi_values}[k] = \text{KalmanFilter}(\{\text{RSSI}_i \mid i \in G\})$$

$$\text{average_rssi}[k] = \frac{1}{|\text{rssi_values}[k]|} \sum_{i \in \text{rssi_values}[k]} \text{RSSI}_i$$

$$\text{normalized_rssi}[k] = \frac{\text{average_rssi}[k] - \text{RSSI_MIN}}{\text{RSSI_MAX} - \text{RSSI_MIN}}$$

Normalisation des distances

Pour chaque balise k dans le groupe G :

$$\text{distances}[k] = \begin{cases} \text{DijkstraPathLength}(\text{graph}, \text{last_visited_beacon}, k) - 1 & \text{si chemin existe} \\ 1000 & \text{sinon} \end{cases}$$

$$\text{normalized_distance}[k] = \frac{\text{distances}[k] - \text{DISTANCE_MIN}}{\text{DISTANCE_MAX} - \text{DISTANCE_MIN}}$$

Calcul du score normalisé

Pour chaque balise k :

$$S_k = w_{\text{RSSI}} \times \text{normalized_rssi}[k] + w_{\text{Count}} \times \text{beacon_count}[k] - w_{\text{Distance}} \times \text{normalized_distance}[k]$$

Sélection du meilleur candidat

Le meilleur candidat est celui avec le score S_k le plus élevé :

$$\text{Meilleur Candidat} = \arg\max_k (S_k)$$

Le score du meilleur candidat est :

$$\text{Score du Meilleur Candidat} = S_{\text{Meilleur Candidat}}$$

En connaissant alors le score du meilleur candidat, on peut alors désormais construire la trace décrivant le déplacement dans le graphe des possibles tels que :

```
13  class PlayerTrace:
14      def __init__(self, has_visited_beacons, last_visited_beacon, last_data_time, temporal_sequence_visited):
15          self.has_visited_beacons = has_visited_beacons
16          self.last_visited_beacon = last_visited_beacon
17          self.last_data_time = last_data_time
18          self.temporal_sequence_visited = temporal_sequence_visited
```

```python
def move_in_network(self):

    for group in self.group_data_based_on_time_window(self.time_interval_of_groups):
        best_candidate, best_candidate_score, _ = self.get_best_candidate(group)
        best_candidate = str(best_candidate)

        print("Best candidate: ", best_candidate, " with a score of ", best_candidate_score)

        if best_candidate_score < self.CONFIDENCE_THRESHOLD:
            continue

        if self.last_visited_beacon != None:
            shortest_path = nx.dijkstra_path(self.model_network.graph,
                                             str(self.last_visited_beacon if self.last_visited_beacon else self.has_visited_beacons[-1]),
                                             best_candidate)
        else:
            shortest_path = [best_candidate]

        self.has_visited_beacons.extend([node for node in shortest_path if node not in self.has_visited_beacons])

        if ((self.last_visited_beacon == None or self.last_visited_beacon != best_candidate) or self.debug):
            print("Moving from ", self.last_visited_beacon, " to ", best_candidate, " at ", group[-1][DATE_KEY], " with a score of ", rou
            date_to_string = group[-1][DATE_KEY]
            self.temporal_sequence_visited[str(date_to_string)] = {
                "from": self.last_visited_beacon,
                "to": shortest_path[1:] if len(shortest_path) > 1 else [best_candidate],
                "score": best_candidate_score if not self.debug else _
            }

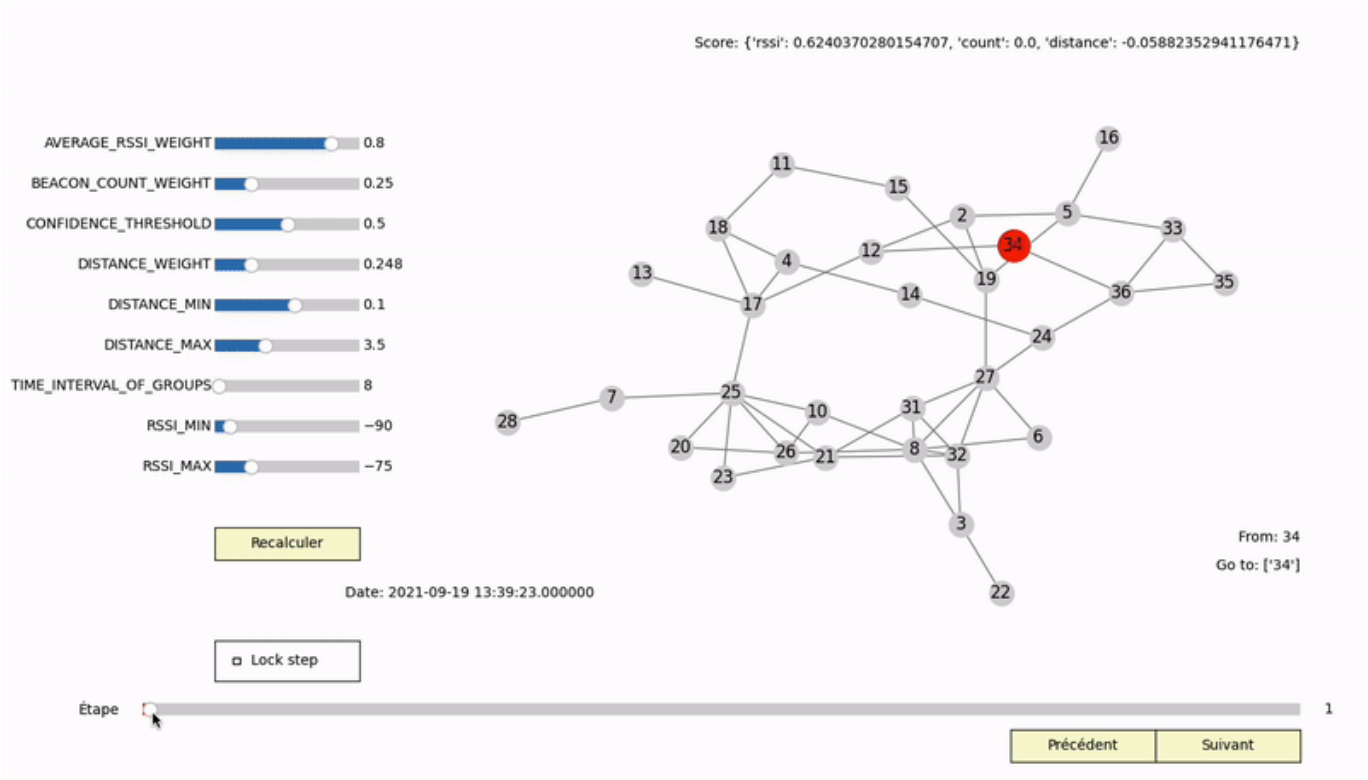
        self.last_visited_beacon = best_candidate

    print("Last visited: ", self.last_visited_beacon, self.has_visited_beacons, self.last_data_time)

    return PlayerTrace(self.has_visited_beacons, self.last_visited_beacon, self.last_data_time, self.temporal_sequence_visited)
```

On peut alors avoir la trace « temporal_sequence_visited » où chaque clé permet de savoir le timestamp à laquelle le déplacement d'un joueur a été enregistré et permet d'avoir des données tel que d'où il est parti et le chemin qu'il a pris pendant son déplacement à travers les différents nœuds. Grâce à ça on peut alors reconstruire la trace d'un joueur très facilement. On peut également savoir quel nœud le joueur a visité très facilement en consultant « has_visited ». On sait également la dernière position du joueur grâce à « last_visited » (l'algorithme s'en sert notamment pour savoir le point de

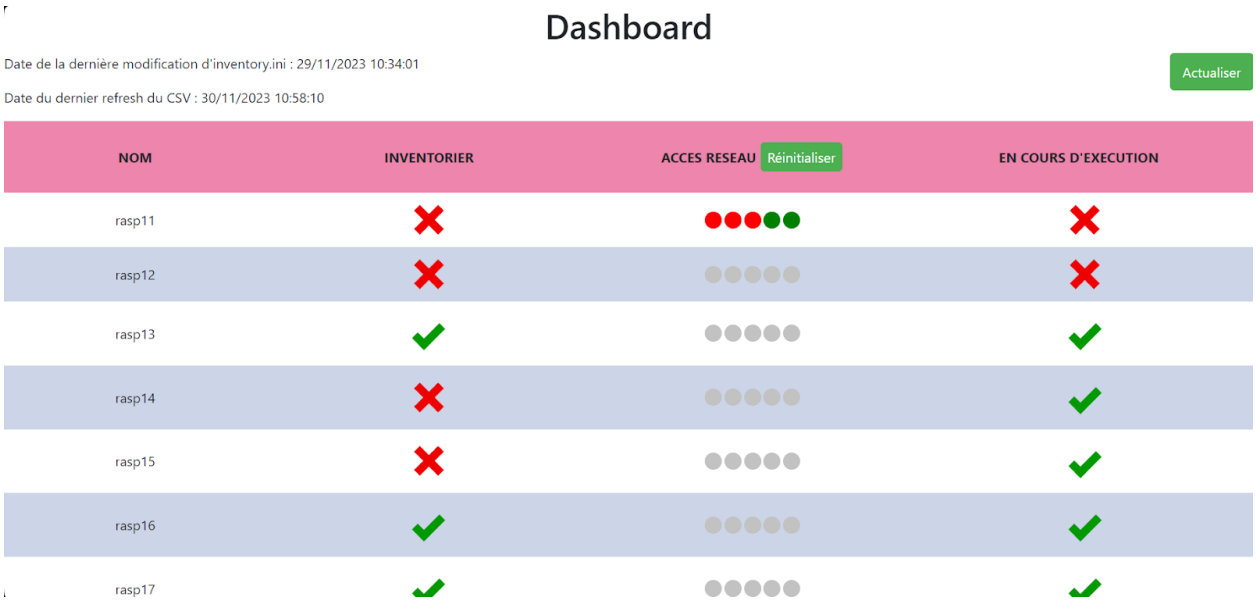
départ du prochain déplacement du joueur très rapidement). Ces traces peuvent être alors assemblées et présentées dans un outil permettant de configurer et comprendre comment l'algorithme marche dans un petit outil de visualisation des données telles que :



Cet outil retrace alors les déplacements du signal BLE dans l'environnement à partir des données brutes permettant une calibration et une visualisation post-analyse, cela permet également de débugger et rendre compte de l'efficacité de l'algorithme, cela n'était pas réellement une demande mais m'a permis de débugger et comprendre comment l'algorithme réagissait à des changements dans le code de manière plus précise et plus agréable qu'un « temporal_sequence_visited » brut.

10 Configurer, gérer, orchestrer et visualiser l'environnement depuis un tableau de bord

10.1 Problèmes rencontrés avec le tableau de bord existant



Lors de la SAE réalisée à l'IUT, un tableau de bord avait été développé, mais il présentait plusieurs limitations importantes :

Stockage et gestion des données :

- Le tableau de bord s'appuyait sur des fichiers de configuration Ansible pour déclarer les Raspberry Pi nécessaires à l'orchestration.
- Ces fichiers étaient écrits en dur, ce qui rendait toute interaction difficile et nécessitait des modifications manuelles dans des fichiers texte, souvent situés dans des emplacements complexes à trouver.

Suivi de l'état des Raspberry Pi :

- Le tableau de bord devait récupérer des informations de connexion et l'état des Raspberry Pi, tels que :

- La connexion réseau (est-ce que le Raspberry est en ligne ?)
- Le statut des différents services embarqués

Pour cela, Ansible était utilisé pour établir des connexions SSH et exécuter des commandes de diagnostic (comme des pings), avec les résultats stockés dans des fichiers texte.

Performances et stabilité :

- Les connexions SSH sont lentes et Ansible n'est pas conçu pour des opérations rapides. Cela entraînait des délais importants et des résultats instables.
- Le tableau de bord était programmé sous Symfony PHP, rendant l'ensemble de la stack lourde et difficile à maintenir.

10.2 Nouvelle solution proposée

Pour résoudre ces problèmes, j'ai décidé de concevoir une API légère que chaque Raspberry Pi pourrait embarquer telle que :

```python
from flask import Flask, request
import subprocess

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

@app.route('/status')
def service_status():
    service_names = request.args.getlist('service_name')
    statuses = {}
    for service_name in service_names:
        status_result = subprocess.run(['systemctl', 'is-active', service_name], capture_output=True, text=True)
        # Store the service status in a dictionary using the service name as the key.
        statuses[service_name] = status_result.stdout.strip()
    return statuses

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Cette API permet de récupérer toutes les informations nécessaires via de simples requêtes HTTP « GET ». Voici les avantages de cette approche :

- Les méthodes HTTP « GET » sont beaucoup plus rapides que les connexions SSH, permettant une récupération quasi-instantanée des informations
- L'API permet de connaître l'état des Raspberry Pi et de leurs services, comme le service RaspLogs pour la détection des signaux BLE, assurant ainsi le bon fonctionnement de l'environnement en temps quasi-réel.

10.3 Amélioration de la gestion de l'inventaire

Malgré ces améliorations, il restait un problème avec la gestion de l'inventaire d'Ansible ; Ansible permet de définir une liste d'hôtes dans un fichier texte sur lesquels il va alors établir une connexion SSH permettant de transmettre des tâches/ordres. Cependant cette approche n'est pas dynamique et peut être source de nombreuses erreurs. Pour rendre cette gestion plus dynamique et intuitive, j'ai implémentés les solutions suivantes :

Inventaire dynamique :

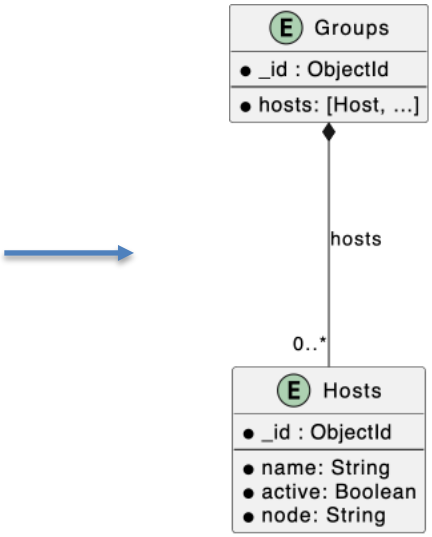
- Au lieu d'utiliser des fichiers texte, l'inventaire des Raspberry Pi est maintenant stocké dans une base de données.
- Cela permet de modifier, ajouter ou supprimer des entrées via une interface de tableau de bord, rendant le processus beaucoup plus intuitif et réduisant les risques d'erreurs.

Identification et activation des Raspberry Pi :

- Chaque Raspberry Pi est enregistré sur le serveur DNS du NUC avec son identité (par exemple, « rasp11.local »), ce qui simplifie l'identification par nom plutôt que par adresse IP.

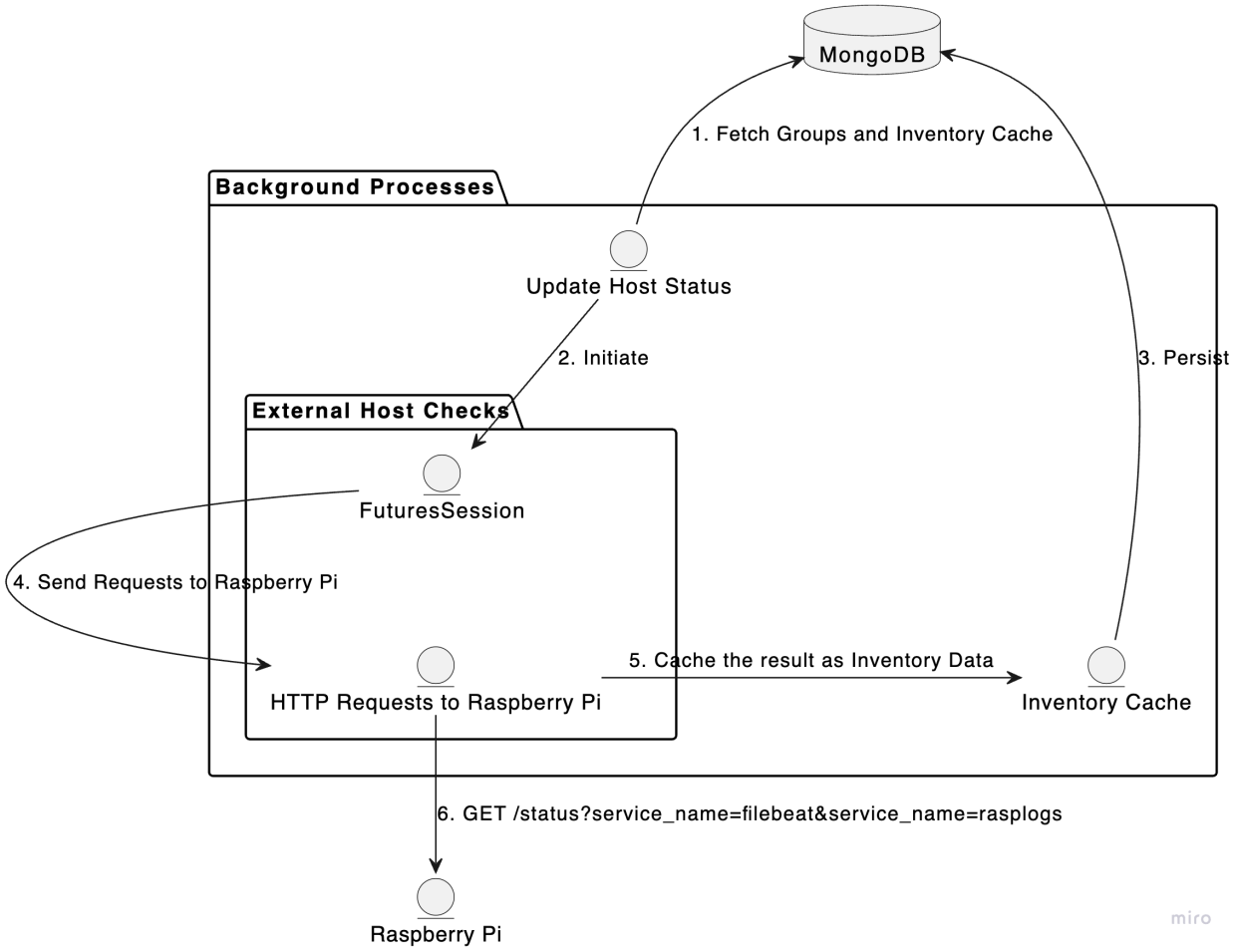
- Une variable active permet de savoir si le Raspberry Pi doit être inclus dans l'orchestration Ansible.
- Une variable node lie chaque Raspberry Pi à un nœud dans le graphe des possibles, permettant de localiser les données envoyées par Logstash.

```
[raspberry]
rasp11.local ansible_ssh_user=admin
rasp12.local ansible_ssh_user=admin
rasp13.local ansible_ssh_user=admin
rasp14.local ansible_ssh_user=admin
rasp15.local ansible_ssh_user=admin
rasp16.local ansible_ssh_user=admin
rasp17.local ansible_ssh_user=admin
rasp18.local ansible_ssh_user=admin
rasp19.local ansible_ssh_user=admin
rasp20.local ansible_ssh_user=admin
rasp21.local ansible_ssh_user=admin
rasp22.local ansible_ssh_user=admin
rasp23.local ansible_ssh_user=admin
rasp24.local ansible_ssh_user=admin
rasp25.local ansible_ssh_user=admin
rasp26.local ansible_ssh_user=admin
rasp27.local ansible_ssh_user=admin
rasp28.local ansible_ssh_user=admin
rasp29.local ansible_ssh_user=admin
rasp30.local ansible_ssh_user=admin
```

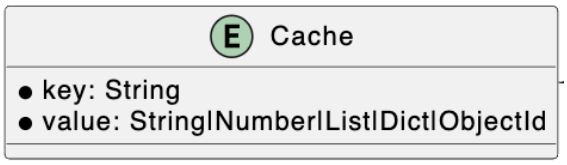


10.4 Création d'une tâche permettant de récupérer automatiquement l'état des Raspberry

Pour améliorer la surveillance des Raspberry Pi, une tâche automatique peut être mise en place pour récupérer régulièrement leur état. Cette tâche en arrière-plan interroge périodiquement tous les Raspberry Pi de l'environnement en effectuant des requêtes vers l'endpoint `/status` de chaque appareil pour obtenir leur état actuel.



Les résultats des requêtes sont ensuite stockés dans une base de données, en conservant un historique des cinq derniers états de chaque Raspberry Pi. Cette structure de données permet également d'ajouter d'autres valeurs à surveiller à l'avenir.




```
{
    _id: ObjectId('6658920f07608d769cf7f365'),
    key: 'inventory',
    value: {
        'rasp13.local': {
            name: 'rasp13.local',
            online: false,
            services: {
                rasplogs: null,
                filebeat: null
            },
            last_pings: [
                false,
                false,
                false,
                false
            ],
            timestamp_last_successful_ping: null
        }
    }
}
```

Grâce à cette tâche, le tableau de bord peut accéder rapidement aux états récents des Raspberry Pi sans devoir attendre que chaque requête GET soit traitée en temps réel. Cela améliore significativement la réactivité et la performance du tableau de bord, offrant une vue quasi-instantanée de l'état de l'environnement.

10.5 Nouvelles demandes concernant le tableau de bord

Afin de répondre aux besoins croissants et d'améliorer l'efficacité et la flexibilité de l'infrastructure, de nouvelles fonctionnalités ont été identifiées pour le tableau de bord :

10.5.1 Gestion de l'inventaire

1. Création, modification et suppression de Raspberry Pi :
    - Permettre aux utilisateurs de créer, modifier et supprimer des entrées pour les Raspberry Pi dans l'inventaire via l'interface du tableau de bord.

- Simplifier la gestion des appareils et assurer que les informations soient toujours à jour.

10.5.2 Gestion du graphe des possibles

2. Création d'un graphe des possibles depuis le tableau de bord :

    - Permettre la création et la gestion d'un graphe des possibles directement depuis le tableau de bord.
    - Utiliser un fichier SVG pour avoir une représentation précise du plan de l'environnement, avec la localisation des différents nœuds et les chemins entre eux.

10.5.3 Gestion des profils

3. Gestion des profils de configuration :

    - Permettre la création de plusieurs profils de configuration d'inventaire et de graphe des possibles.
    - Chaque profil peut correspondre à une configuration spécifique, facilitant les expérimentations dans différents environnements.

10.5.4 Orchestration et gestion des services

4. Lancement des scripts Ansible depuis le tableau de bord :

    - Intégrer la capacité de lancer des scripts Ansible directement depuis le tableau de bord.
        - Création d'un WebSocket permettant la réception de la sortie du script d'Ansible à chaque changement depuis le tableau de bord.
    - Faciliter la gestion des Raspberry Pi, permettant le déploiement de services, la mise à jour de logiciels, etc.

10.5.5 Visualisation des joueurs

5. Visualisation des mouvements des joueurs :

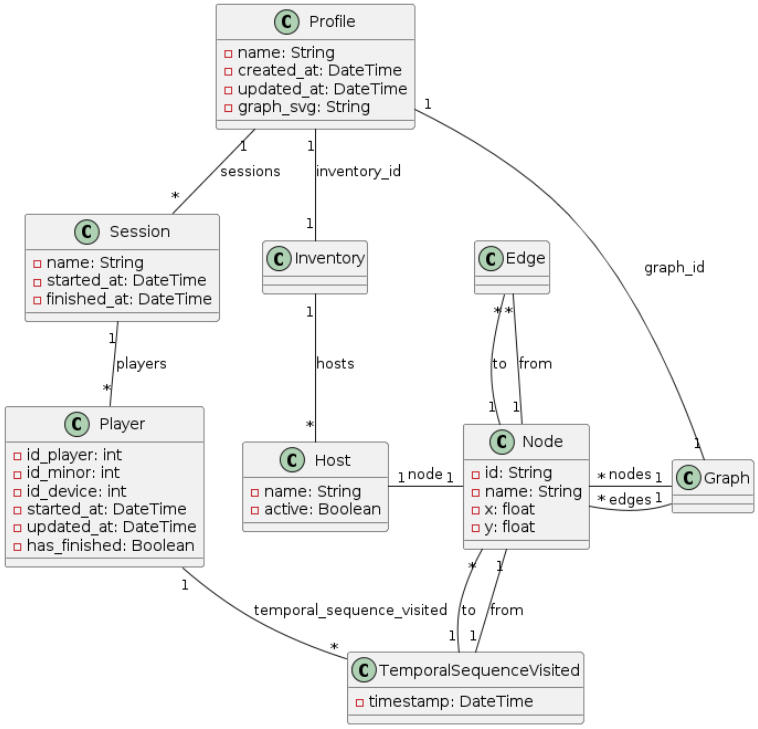
    - Afficher les mouvements des joueurs à travers le graphe des possibles, en utilisant les données de positionnement.
    - Fournir une visualisation en temps réel des déplacements et interactions des joueurs dans l'environnement.

10.5.6 Gestion des sessions

6. Gestion des sessions :

    - Permettre la création et la gestion de sessions, chaque session étant liée à un profil spécifique.
    - Définir la période pendant laquelle l'environnement est actif et collecter les données de tous les joueurs durant cette période.

10.6 Conception des nouvelles fonctionnalités du tableau de bord



10.7 Conception d'une API

Pour communiquer et permettre la récupération de toutes les informations du Tableau de bord, nous devons concevoir une API capable d'interagir non seulement avec le tableau de bord, mais également avec des appareils externes tels que Pepper et Les Aventures de Rose. Cela permettra à ces solutions d'interagir avec notre environnement pour, par exemple, enregistrer un joueur ou récupérer un graphique, etc. Les détails de l'API, incluant toutes les routes, sont répertoriés en annexe.

```python
from flask import request
from flask_restx import Resource, fields

from lib.api import send_error, send_success, create_combined_model
from lib.models import graph_model, error_response, generic_success_response

from modules.database.utils.graphs_utils import update_profile_graph, download_graph_yaml
from modules.database.query.get_active_profile_graph import get_active_profile_graph

from lib.api import api

graph_ns = api.namespace('graph', description='Graph operations')

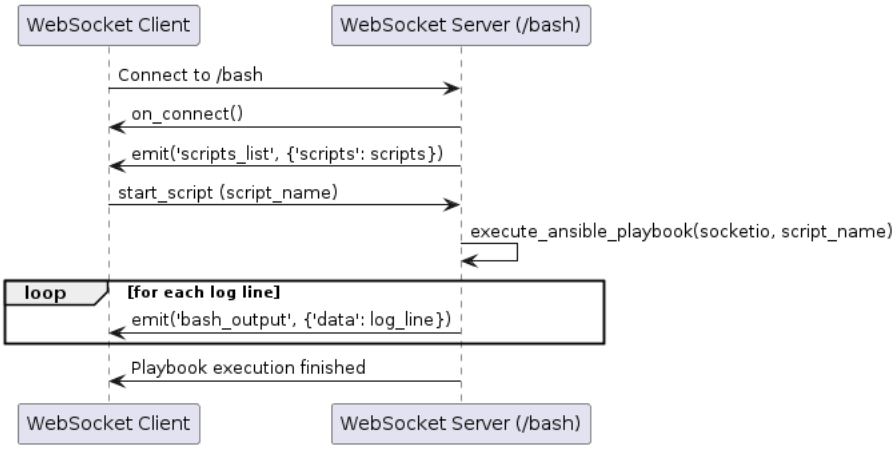
@graph_ns.route('')
@api.response(400, 'Error updating the graph', model=error_response)
@api.response(200, 'Success', model=generic_success_response)
class Graph(Resource):
    @api.doc(description='Retrieve the graph for the active profile')
    @api.marshal_with(create_combined_model(graph_model), code=200, description='Graph retrieved successfully')
    def get(self):
        """Retrieve the graph for the active profile."""
        try:
            graph = get_active_profile_graph()
            return send_success("Graph retrieved successfully", graph)
        except Exception as e:
            return send_error(str(e))
```

L'API est réalisée en Python, tout comme l'infrastructure générale, en utilisant "flask-restx". Cette bibliothèque permet de créer des API avec des interfaces intégrées telles que Swagger, offrant ainsi une documentation complète et la possibilité d'effectuer des appels depuis une page de contrôle. Cela nous permet d'avoir une documentation directe et une vue d'ensemble de l'API.

10.8 Conception d'un WebSocket pour Ansible

Pour permettre la communication entre Ansible et le tableau de bord, une simple route API ne serait pas suffisante. Nous devons utiliser une alternative telle qu'un WebSocket pour implémenter cette fonctionnalité. Un WebSocket permet une communication bidirectionnelle en temps réel entre le serveur et le client, ce qui est essentiel pour recevoir les logs d'Ansible en temps réel. Ces logs permettent de suivre l'avancement des ordres d'orchestration en cours, de savoir quelle tâche est en cours d'exécution, et d'identifier les Raspberry Pi pour lesquels l'orchestration a rencontré des erreurs.

On décrit la communication entre le client et le serveur telle que :



On définit une liste de « scripts » pouvant être exécuté depuis le tableau de bord dans un fichier config.yml tel que :

```yaml
  start:
    name: Start
    description: Start all services on the selected hosts
    playbooks:
      '1':
        path: ansible/playbooks/control-ntp.yml
      '2':
        path: ansible/playbooks/control-script.yml
        vars:
          action: start
```

On définit alors la liste des playbooks à exécuter dans l'ordre chronologique et des variables pour chaque playbook si besoin, on peut par exemple prendre en exemple le script de control-ntp qui définit ces actions :

```yaml
- name: On récupère la date et l'heure actuelle du NUC
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Obtenir la date et l'heure actuelle
      command: date "+%Y-%m-%d %T"
      register: nuc_datetime

- name: On configure la date et l'heure sur les Raspberry Pi
  hosts: raspberry
  become: yes
  gather_facts: false
  vars:
    datetime_to_set: "{{ hostvars['localhost']['nuc_datetime'].stdout }}"
  tasks:
    - name: Mettre à jour la date et l'heure
      command: sudo date -s "{{ datetime_to_set }}"
```

On peut également prendre un autre exemple permettant les mises à jour d'un projet git défini sur les Raspberry :

```yaml
update-webserver:
  name: Update Webserver
  description: Update the Webserver service on the selected hosts
  playbooks:
    '1':
      path: ansible/playbooks/git-pull.yml
      vars:
        gitlab_access_token: true
        repo_url: forge.iut-larochelle.fr/ptuts5-equipe4/rasp/rasp-webserver.git
        slug: rasp-webserver
        branch: main
```

Depuis le tableau de bord on choisit alors un script ce qui envoie au WebSocket Serveur le script puis éxecutera un par un les playbooks en utilisant la librairie « ansible_runner », le Websocket Serveur renverra alors chaque nouvelle ligne de log pour ce script associé au client afin que le client puisse observer et constater l'exécution des ou du playbook en cours tels que :

```
    for key in sorted(playbook_group['playbooks'].keys()):
        playbook = playbook_group['playbooks'][key]
        playbook_path = playbook['path']
        playbook_vars = playbook.get('vars', {})

        # Check if gitlab_access_token is to true
        if playbook_vars.get('gitlab_access_token', False):
            gitlab_access_token = os.getenv('GITLAB_ACCESS_TOKEN')
            if not gitlab_access_token:
                error_message = 'Gitlab access token not set.\n'
                socketio.emit('bash_output', {'data': error_message}, namespace='/bash')
                create_document_ansible_log(playbook_key, start_time, error_message)
                return

            playbook_vars['gitlab_access_token'] = gitlab_access_token

        if not os.path.isfile(playbook_path):
            error_message = 'Playbook does not exist.\n'
            socketio.emit('bash_output', {'data': error_message}, namespace='/bash')
            create_document_ansible_log(playbook_key, start_time, error_message)
            return

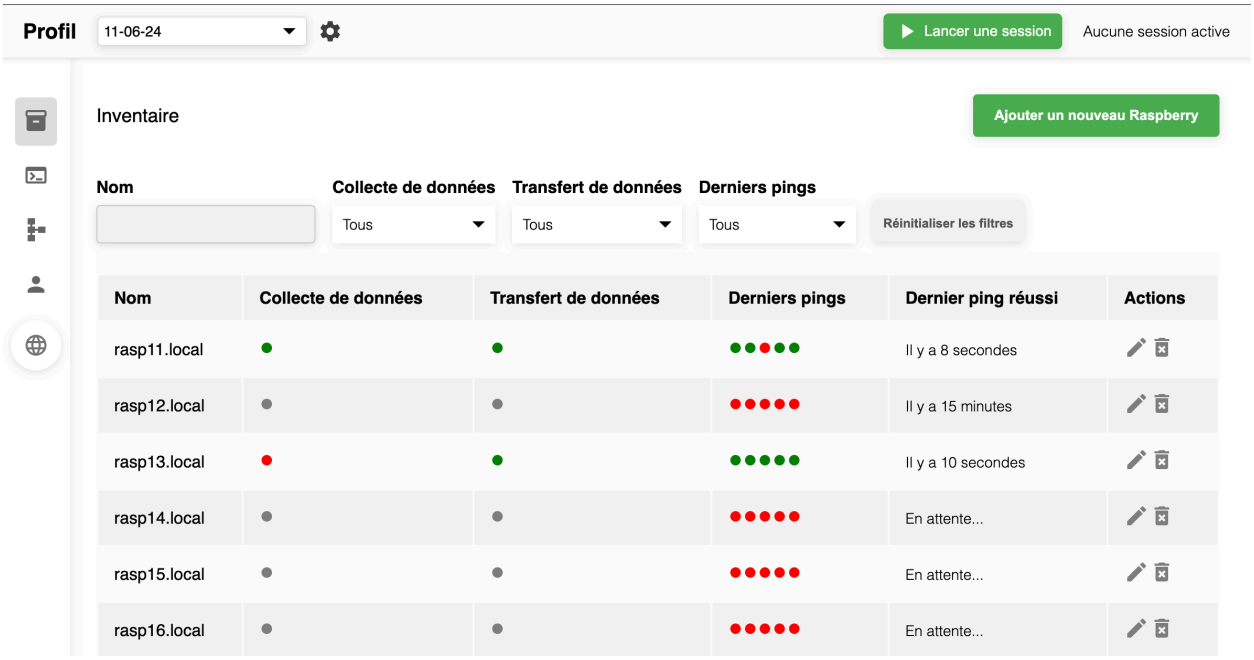
        try:
            r = ansible_runner.run(
                private_data_dir='.',
                playbook=playbook_path,
                inventory=inventory,
                extravars=playbook_vars,
                event_handler=custom_event_handler,
                status_handler=finished_callback
            )

        except Exception as e:
            error_message = f"An error occurred: {str(e)}\n"
            socketio.emit('bash_output', {'data': error_message}, namespace='/bash')
            create_document_ansible_log(playbook_key, start_time, error_message)
            return

    socketio.emit('bash_output', {'data': '\n\nPlaybook execution finished.\n'}, namespace='/bash')
```

10.9 Détails des nouvelles fonctionnalités

10.9.1 Gestion de l'inventaire

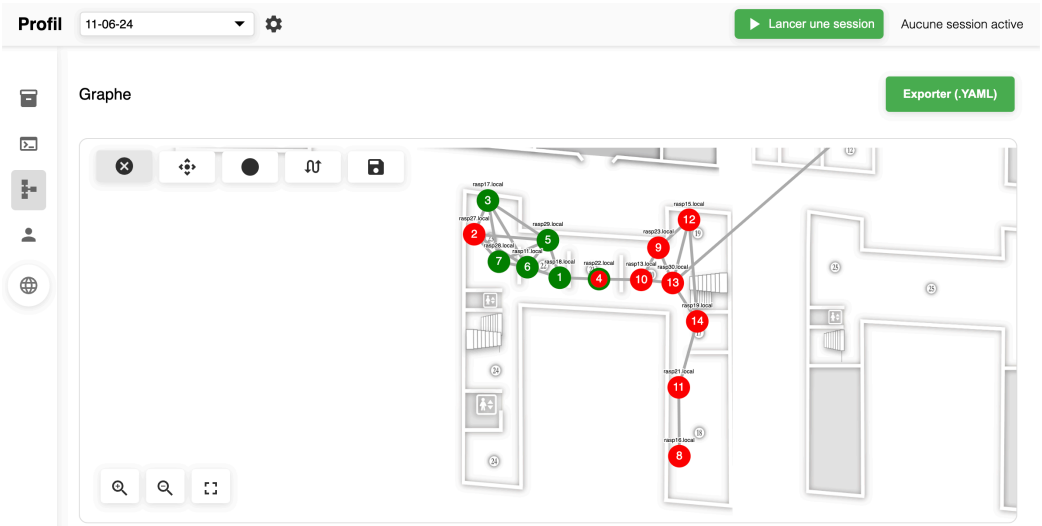


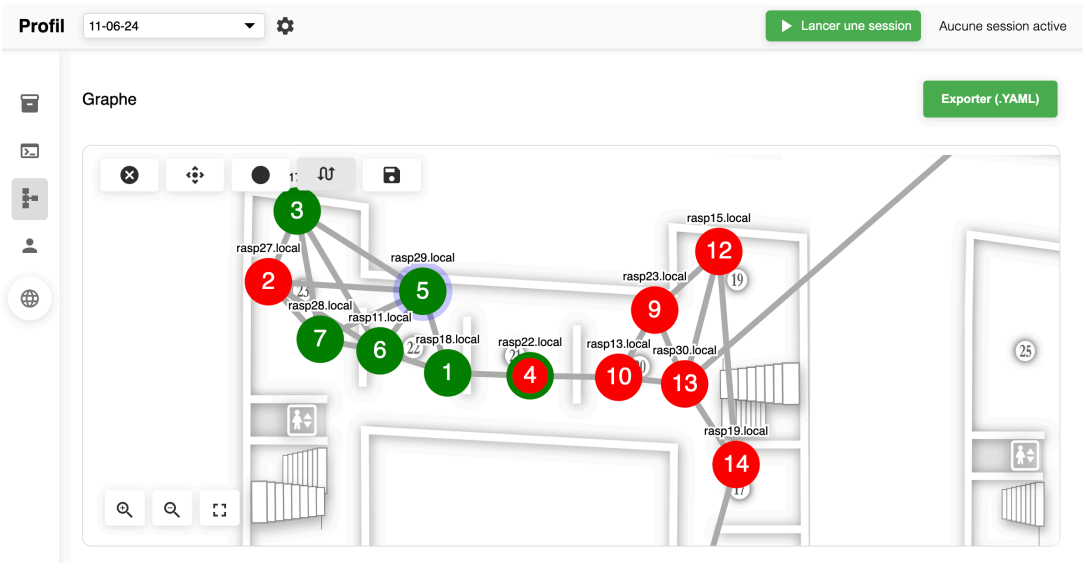
- Une interface intuitive permettant de gérer l'inventaire des Raspberry Pi.
- Facilite l'ajout, la modification et la suppression de dispositifs sans avoir à manipuler des fichiers texte.

10.9.2 Gestion du graphe des possibles :

- Utilisation d'un fichier SVG pour définir le plan de l'environnement.
- Manipulation du plan sous format SVG tel que le zoom et déplacement/défilement.
- Possibilité de positionner les différents nœuds et de définir les chemins entre eux, offrant une représentation visuelle claire de l'espace du musée

- Surveillance de l'état de l'environnement (raspberry en ligne, hors ligne et/ou état de la remontée des données)

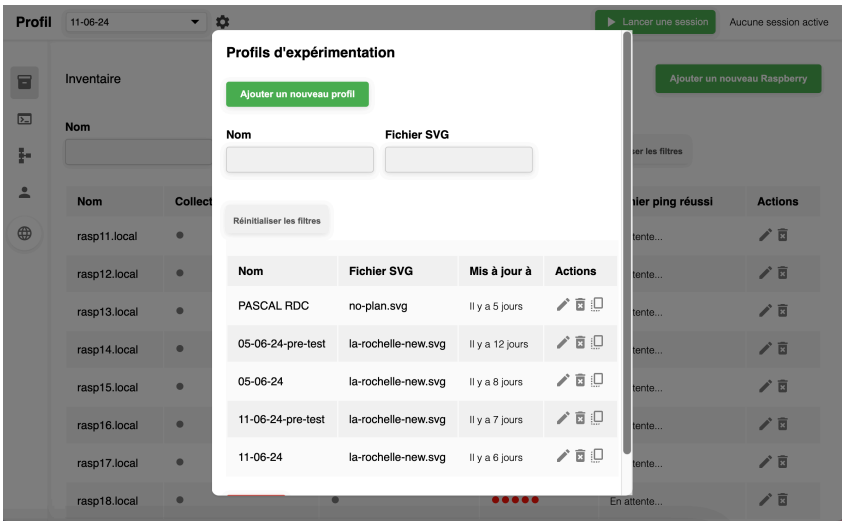


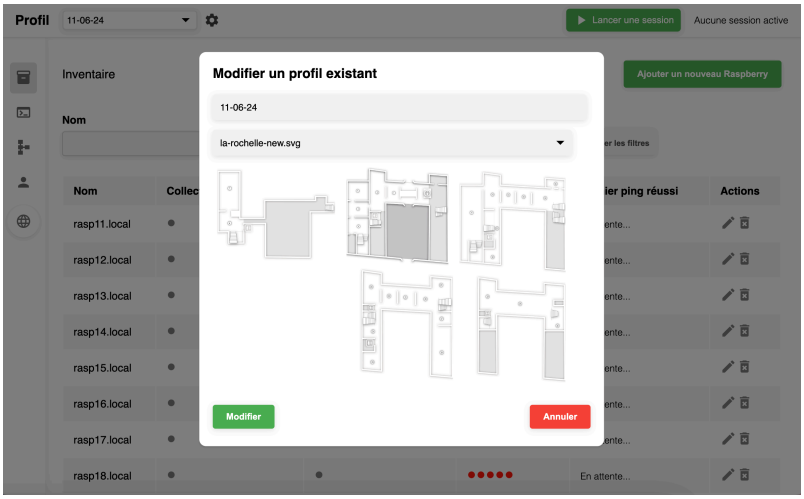


10.9.3 Gestion des profils

- Chaque profil peut contenir une configuration d'inventaire et de graphe des possibles spécifique.
- Contient également l'historique des sessions associé au profil.

- Permet de basculer facilement entre différentes configurations pour des expérimentations dans divers environnements.

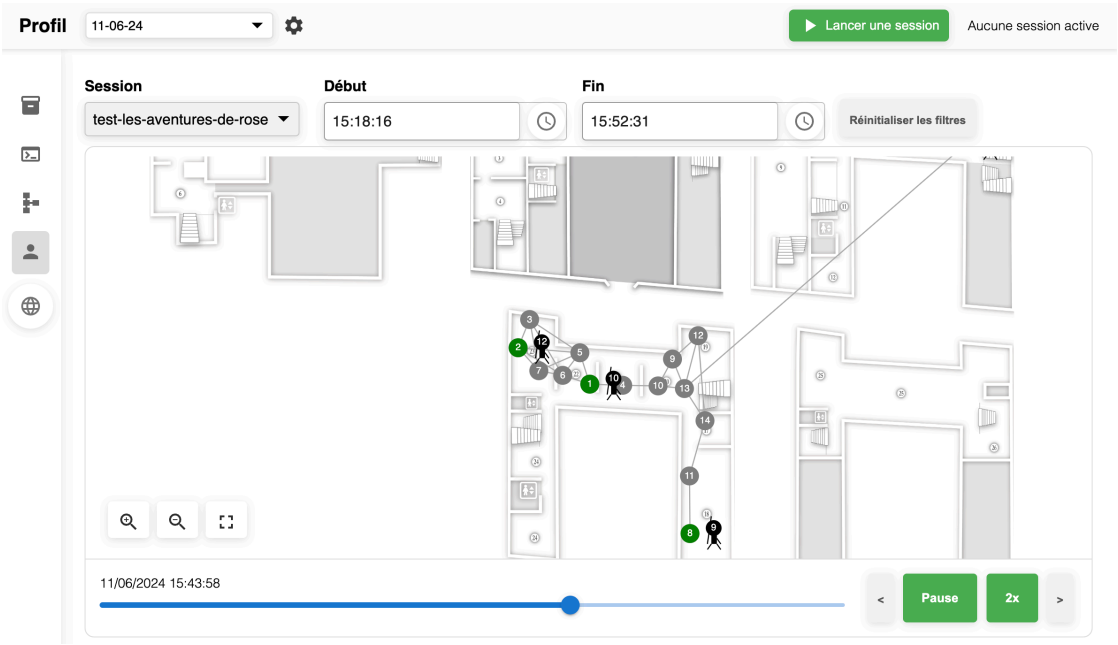




10.9.4 Visualisation des joueurs

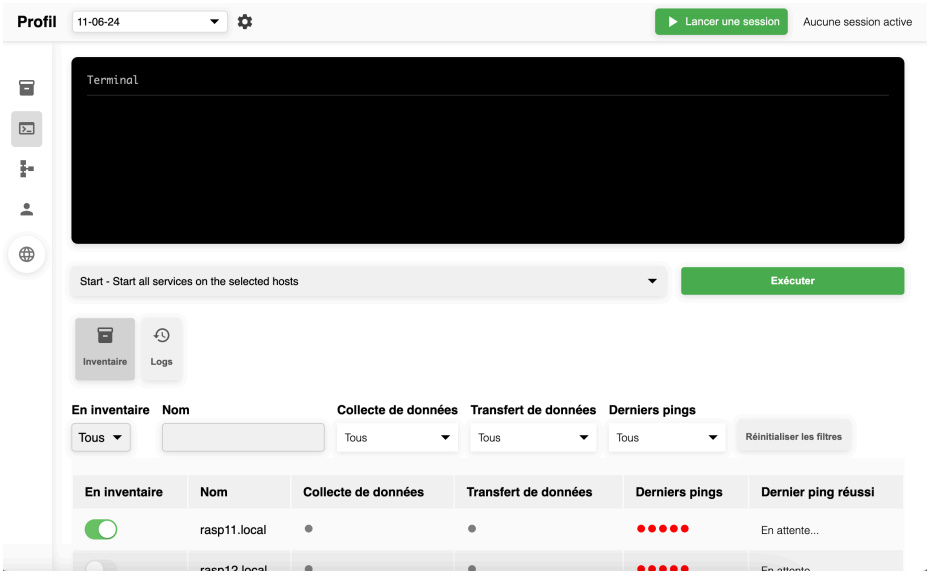
- Affichage en temps réel des déplacements des joueurs sur le graphe des possibles.
- Suivi des interactions des joueurs avec l'environnement.

- Stockage des données de chaque joueur pour une analyse détaillée de leurs déplacements durant une session.



10.9.5 Orchestration et gestion des services

- Intégration directe de la gestion Ansible, permettant de lancer des services, effectuer des mises à jour et gérer les Raspberry Pi depuis le tableau de bord.



Ces nouvelles fonctionnalités visent à rendre le tableau de bord plus complet, flexible et facile à utiliser, tout en introduisant la gestion et la visualisation de l'environnement et des interactions des joueurs.

11 Exemple d'intégration

On peut alors déjà se demander en l'état comment l'environnement pourrait s'intégrer à d'autre interface.

11.1 Les Aventures de Rose

Pendant mon stage j'ai pu être en interaction avec l'équipe du projet des Aventures de Rose. Ce Serious Game se définit tel que :

> Enquêtes au musée
>
> Les aventures de Rose est un « *Serious Game* » conçu pour accueillir des classes de cycle 3 (CM1, CM2, 6e) au musée et faire découvrir aux élèves de façon ludique les collections et le patrimoine. Répartis en groupes de 4 à 6 élèves, les enfants partent en autonomie dans le musée, guidés par Rose. Grâce à la tablette, les enfants sont envoyés dans différents espaces du musée en quête de plusieurs indices. C'est en collectant ces indices qu'ils seront amenés, une fois réunis en classe entière, à devoir tous collaborer pour résoudre un mystère.[5]



[5] https://www.alienor.org/publications/373-les-aventures-de-rose-jeu-serieux-a-destination-des-classes

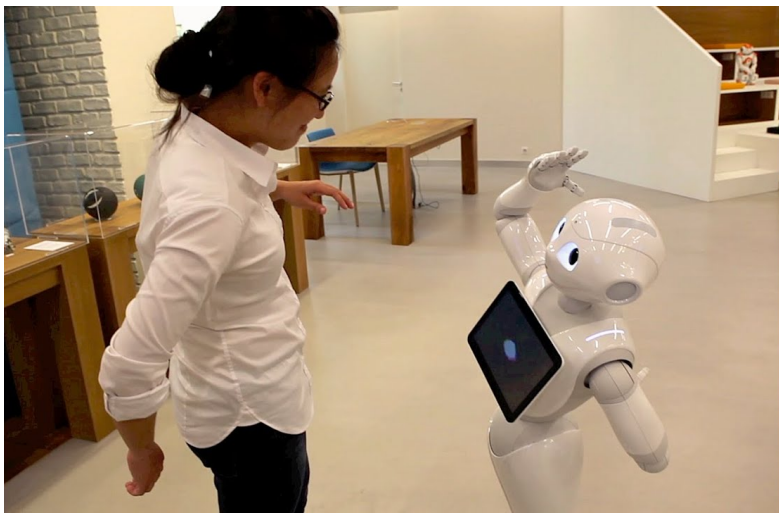
Nous avons réalisé des tests de notre système de micro-localisation en conditions réelles au Muséum d'Histoire Naturelle de La Rochelle, au deuxième étage. Cela nous a permis de recueillir des données réelles et de vérifier la fiabilité de notre système à travers une phase de jeu. Nous avons également effectué une post-analyse pour vérifier l'exactitude des données traitées.

Quatre membres de l'équipe des Aventures de Rose ont testé une phase de leur jeu. Nous avons installé des Raspberry Pi sur tout le deuxième étage pour observer les mouvements des participants équipés de badges BLE iBeacon.

Cette expérience a été fructueuse, nous permettant de collecter des données concrètes sur les déplacements de plusieurs joueurs en simultané dans l'environnement. Cela démontre la flexibilité et la pertinence de notre système, ainsi que la qualité des post-analyses possibles.

11.2 Le robot Pepper

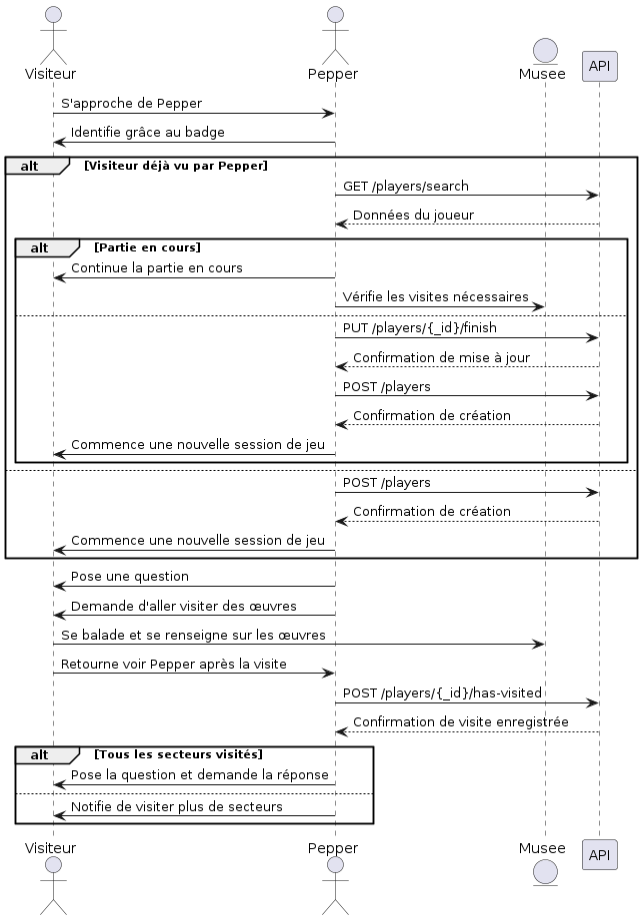
Un exemple concret autre est l'utilisation du robot Pepper au Musée de La Rochelle lors d'événements culturels destinés aux plus jeunes. Ce petit robot humanoïde permet des interactions telles que la reconnaissance faciale, la reconnaissance vocale et est capable de répondre à des interactions sociales humaines basiques. Ce système peut être intégré dans différents jeux ou environnements pour interagir avec des humains. Ces



robots sont, par exemple, utilisés dans certains pays dans des hôpitaux ou à l'accueil de certaines entreprises. Ils sont déjà implémentés dans certains jeux au sein du Muséum de La Rochelle.

Actuellement, le jeu du robot Pepper au musée est simple :

- Le robot Pepper pose une question à un visiteur.
- Le visiteur cherche la réponse dans le musée.
- Lors de sa prochaine visite près du robot Pepper, le robot lui demande s'il a trouvé la réponse à la question.
    - Si oui, Pepper lui propose de répondre à la question et, en fonction de la réponse (correcte ou incorrecte), Pepper interagit avec le visiteur de différentes manières.



Cependant, cette phase de jeu présente une faille : la réponse à la question peut être donnée par chance ou par le biais d'une source externe (par exemple, quelqu'un révélant la bonne réponse). Nous pouvons donc intégrer notre environnement dans cette phase de jeu pour vérifier si le visiteur s'est bien rendu dans la zone où se trouve la réponse à la question.

12 Introspection et bilan personnel global

12.1 Un projet complexe par sa diversité

Mon stage au sein du Laboratoire Informatique, Image et Interaction (L3i) de La Rochelle Université a été une expérience extrêmement enrichissante et formatrice. Le projet de micro-localisation en intérieur, auquel j'ai contribué, avait pour objectif de créer un environnement technologique permettant de suivre les visiteurs du Muséum d'histoire naturelle de La Rochelle en temps quasi réel. Ce projet, bien que non encore exploité pour des expériences interactives et ludiques, pose les bases technologiques nécessaires pour de futures applications éducatives et interactives.

Construire un système de micro-localisation est loin d'être une tâche simple. Cela exige une intégration minutieuse de diverses compétences techniques et une compréhension approfondie des défis inhérents à la gestion et à l'analyse de données en temps réel. Travailler sur ce projet m'a permis d'explorer et de maîtriser des domaines variés, tels que l'Internet des Objets (IoT), le développement d'API, et l'utilisation d'outils d'orchestration comme Ansible. Chaque aspect du projet nécessitait une attention particulière et une adaptation constante aux contraintes spécifiques de l'infrastructure et des technologies employées.

12.2 Méthodologie de travail

Durant ce stage, j'ai appliqué une méthodologie de travail rigoureuse et structurée, inspirée des pratiques agiles. J'ai appris à gérer efficacement mon temps et mes tâches, en utilisant des outils de gestion de projet pour suivre l'avancement du travail et assurer une bonne communication avec mon maître de stage, M. Cyril Faucher, et mon enseignante tutrice, Mme. Annick Lassus. Les réunions régulières de suivi avec M. Cyril Faucher accompagné de M. Damien Mondou et M. Jérémy Richard m'ont aidé à rester aligné sur les objectifs du projet et à recevoir des retours constructifs pour améliorer continuellement mon travail.

12.3 Compétences techniques acquises

Développement Full-Stack et DevOps IoT : J'ai renforcé mes compétences en développement full-stack en travaillant sur l'intégration et la gestion de données IoT. J'ai appris à configurer et déployer des Raspberry Pi pour collecter des données via des badges iBeacon BLE, et à utiliser FileBeat et Logstash pour centraliser et traiter ces données.

Orchestration avec Ansible : L'automatisation des mises à jour et de la gestion des Raspberry Pi grâce à Ansible a été une compétence clé acquise durant ce stage. J'ai pu constater l'efficacité de cet outil dans la gestion de grandes infrastructures et l'importance de l'automatisation pour maintenir un environnement cohérent et à jour.

Développement d'API et Gestion de Bases de Données : En développant une API en Python avec Flask et en utilisant MongoDB pour le stockage des données, j'ai pu améliorer mes compétences en développement backend. J'ai appris à créer des interfaces de programmation robustes et sécurisées, essentielles pour la communication entre les différents composants du système.

12.4 Collaborations et interactions

Travailler au L3i m'a permis d'interagir avec de nombreux chercheurs et développeurs, ce qui a été extrêmement bénéfique pour mon développement professionnel. Les échanges avec les membres de l'équipe ont enrichi ma compréhension des différentes approches et solutions possibles pour les défis techniques rencontrés.

13 Conclusion finale

Au terme de mon stage au sein du Laboratoire Informatique, Image et Interaction (L3i) de l'Université de La Rochelle, j'ai pu enrichir mes compétences en développement et intégration de systèmes de micro-localisation pour des environnements fermés. Ce projet a été une opportunité unique de travailler sur des technologies diverses, comme les badges BLE, les Raspberry Pi, et des outils d'orchestration tels qu'Ansible. J'ai également acquis une solide expérience dans le développement d'API et la gestion de bases de données NoSQL avec MongoDB.

Ce stage m'a permis de comprendre les défis liés à la collecte et à l'analyse de données en temps quasi réel, ainsi qu'à l'orchestration et à la gestion d'une infrastructure IoT. Les compétences acquises lors de ce projet seront inestimables pour ma future carrière en informatique, en particulier dans les domaines du développement full-stack et de l'IoT.

Je tiens à remercier mon maître de stage, M. Cyril Faucher, M. Mondou Damin et M. Jérémy Richard, pour leurs encadrements et leurs conseils précieux tout au long de cette expérience, ainsi que Mme. Annick Lassus, mon enseignante tutrice, pour son soutien. Grâce à cette expérience, j'ai pu m'immerger dans un projet innovant et complexe, tout en contribuant dans la valorisation des contenus culturels par le numérique.

14 Page de résumé en Français et Anglais

14.1 Résumé en Français :

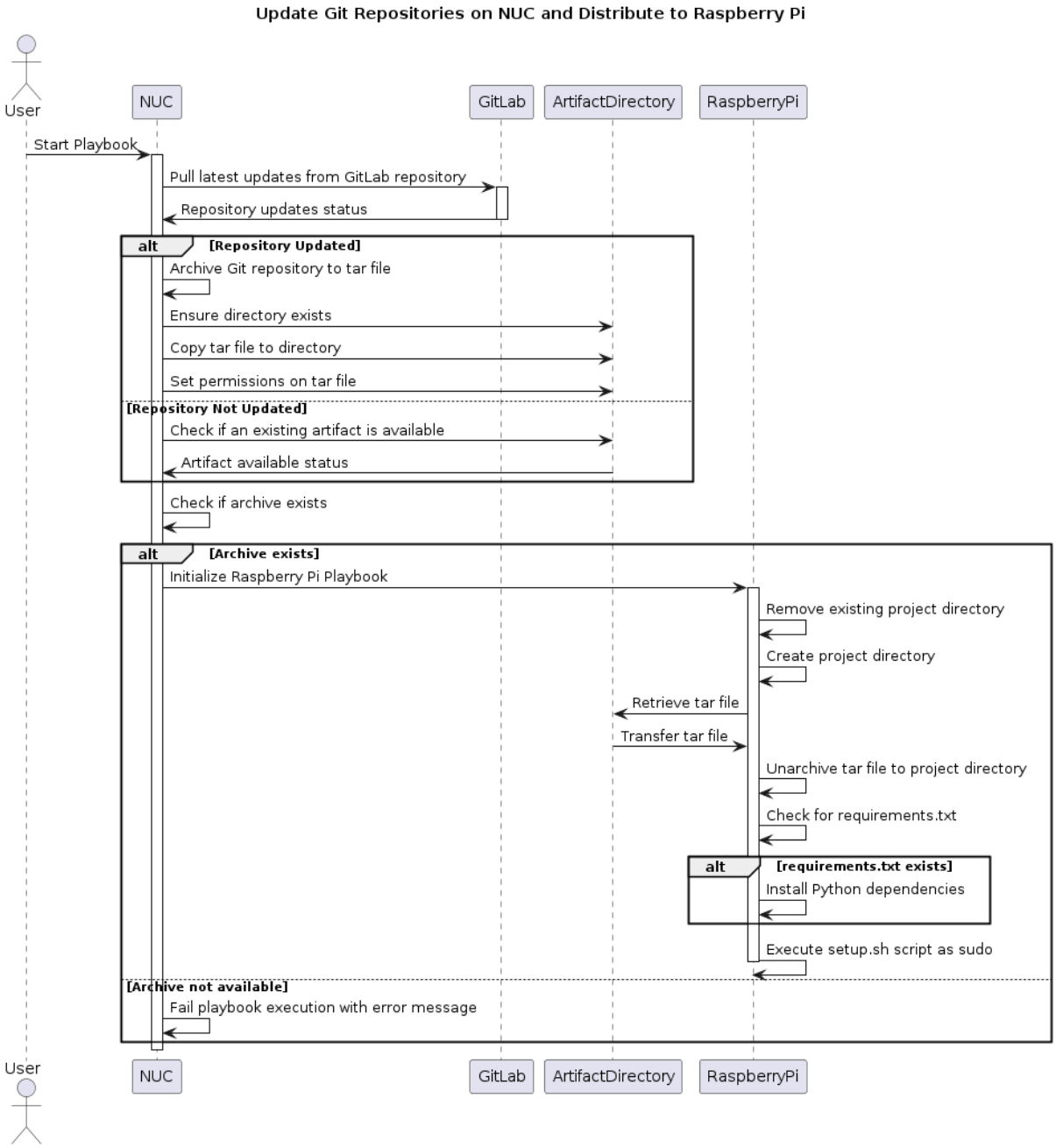
Ce rapport présente le travail réalisé lors de mon stage de troisième année du Bachelor Universitaire de Technologie en Informatique au sein du Laboratoire Informatique, Image et Interaction (L3i) de l'Université de La Rochelle. Le projet visait à mettre en place une solution complète de micro-localisation en quasi-temps réel, permettant une localisation précise des visiteurs dans des espaces fermés comme les musées. Pour ce faire, différentes technologies ont été utilisées, notamment les badges BLE (Bluetooth Low Energy), les micro-ordinateurs Raspberry Pi, ainsi que les outils FileBeat et Logstash pour la collecte et le traitement des données. Ansible a été employé pour l'orchestration et l'automatisation des mises à jour des dispositifs. De plus, j'ai développé des API en Python et géré une base de données MongoDB pour centraliser et analyser les données collectées.

14.2 Résumé en Anglais :

This report presents the work carried out during my third-year internship of the University bachelor's degree in computer science at the Laboratory of Computer Science, Image and Interaction (L3i) of La Rochelle University. The project involved setting up a near real-time micro-location complete solution, enabling precise visitor localization in closed spaces such as museums. To achieve this, various technologies were employed, including BLE (Bluetooth Low Energy) badges, Raspberry Pi micro-computers, and tools like FileBeat and Logstash for data collection and processing. Ansible was used for orchestration and automation of device updates. Additionally, I developed APIs in Python and managed a MongoDB database to centralize and analyze the collected data.

15 Annexe

15.1 Orchestration du déploiement de mise à jour grâce à Ansible sur un parc de Raspberry d'une taille n.



15.2 Détection des signaux BLE et lecture des données publicitaires

Ce projet a été réalisé précédemment dans une SAE (Situation d'Apprentissage Évalué), lié à la demande actuelle citée pendant ce rapport et utilisée actuellement dans l'environnement de détection.

Afin de détecter les signaux BLE on utilise alors le module Bluetooth nativement proposé par le Raspberry Pi Zero, ce module est complétement compatible avec les différents module Python tel que « bluepy » cela s'est alors intégré relativement facilement.

```
class BeaconScanner(DefaultDelegate):
    """
    A class for scanning and detecting iBeacons using Bluetooth Low Energy (BLE) on Linux.

    Methods:
    - __init__(): Initializes the BeaconScanner object and prints a message indicating that Bluetooth
        is enabled and searching for devices.
    - handleDiscovery(dev, isNewDev, isNewData): Overrides the DefaultDelegate's handleDiscovery method to handle the discovery
        of new devices and data. If a new iBeacon is detected,
        it is added to the beacons cache dictionary
        and a message is printed indicating that an iBeacon has been detected.
        If an existing iBeacon is detected, its RSSI is updated and a message is printed indicating that
        the iBeacon has been updated.
    """

    def __init__(self):
        print("Linux: Bluetooth est activé. Recherche des périphériques...")
        DefaultDelegate.__init__(self)

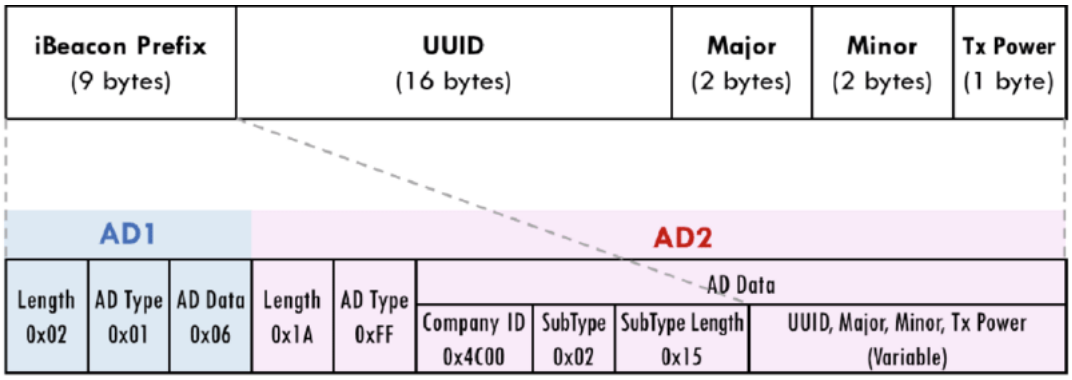
    def handleDiscovery(self, dev, isNewDev, isNewData):
        if isNewDev or isNewData:
            for (adtype, desc, value) in dev.getScanData():
                if desc == "Manufacturer":
                    raw_data = value
                    b = utils.raw_data_to_ibeacon(raw_data, dev.rssi)

                    if b is None:
                        return

                    beacon = beacons.get_beacon(b.get_uuid())
                    if beacon is None:
                        beacons.add_beacon(b)
                    else:
                        beacon.change_rssi(dev.rssi)


if __name__ == "__main__":
    scanner = Scanner().withDelegate(BeaconScanner())
    while True:
        try:
            devices = scanner.scan(10.0, passive=True)
        except Exception as e:
            print(f"Une erreur est survenue : {e}")
```

La lecture des données publicitaires par la suite reste elle aussi assez facile nécessitant seulement de comprendre la structure de la donnée publicitaire telle que :




```python

def raw_data_to_ibeacon(hex_data, rssi):
    """
    Converts raw hexadecimal data to an iBeacon object.

    Parameters:
    hex_data (str): The raw hexadecimal data string.
    rssi (int): The RSSI value associated with the iBeacon.

    Returns:
    iBeacon object or None: The iBeacon object if conversion is successful, else None.
    """

    try:
        # Validate the length of hex_data before processing
        if len(hex_data) < 50:
            return None

        # Extracting identifier and UUID
        identifier = hex_data[:8]
        uuid = hex_data[8:40]

        # Converting major and minor values from hex to int
        major = int(hex_data[40:44], 16)
        minor = int(hex_data[44:48], 16)

        # Check against the major filter signal setting
        if major != MAJOR_FILTER_SIGNAL:
            return None

        # Convert the TX power from hex to signed decimal
        tx_power_hex = hex_data[48:50]
        tx_power = int(tx_power_hex, 16)
        if tx_power_hex[0] > "7":  # Adjust for signed values
            tx_power -= 256

        # Create and return the iBeacon object
        beacon = ibeacon.iBeacon(identifier, uuid, major, minor, tx_power)
        beacon.change_rssi(rssi)
        return beacon

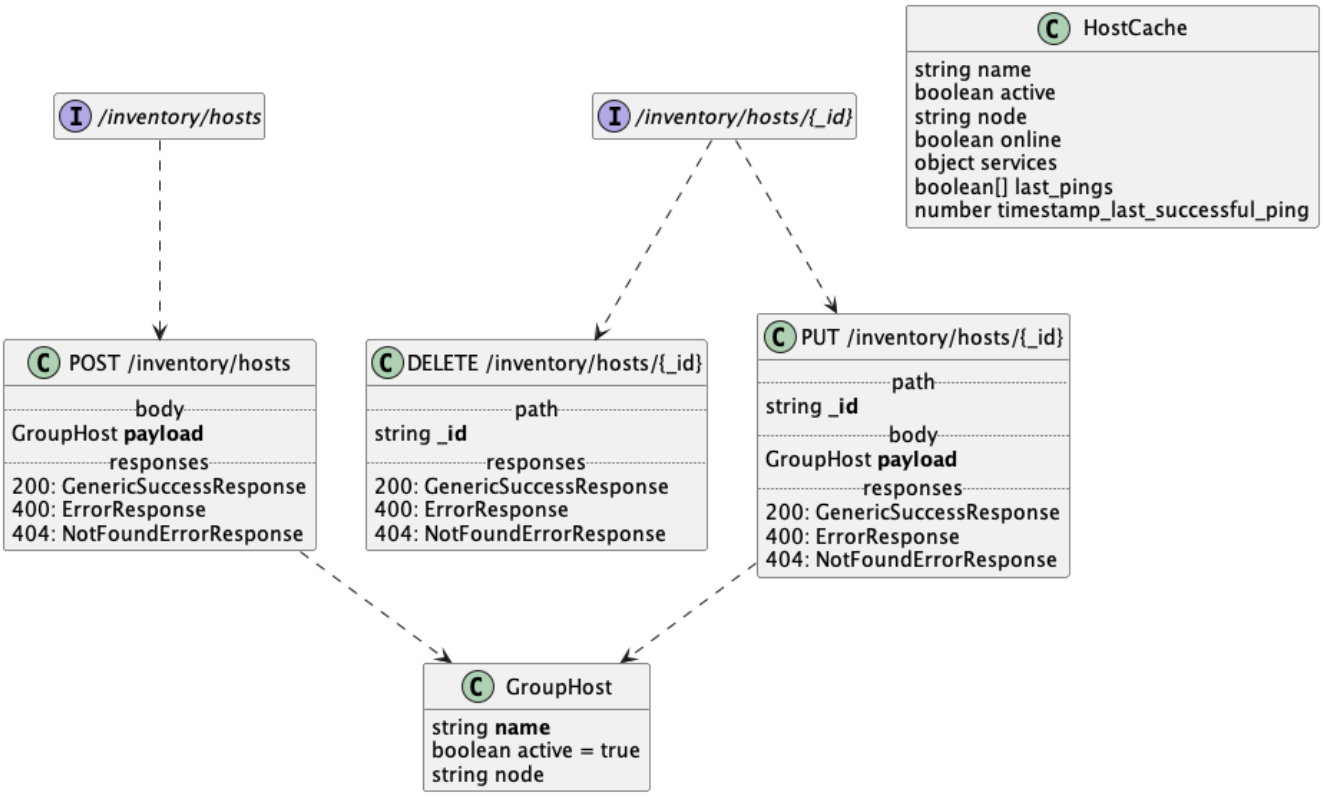
    except Exception as e:
        return None
```

Par la suite on enregistre ces données dans un fichier « data.csv » afin de la traiter et de l'envoyer plus tard grâce à FileBeat vers un serveur de centralisation.

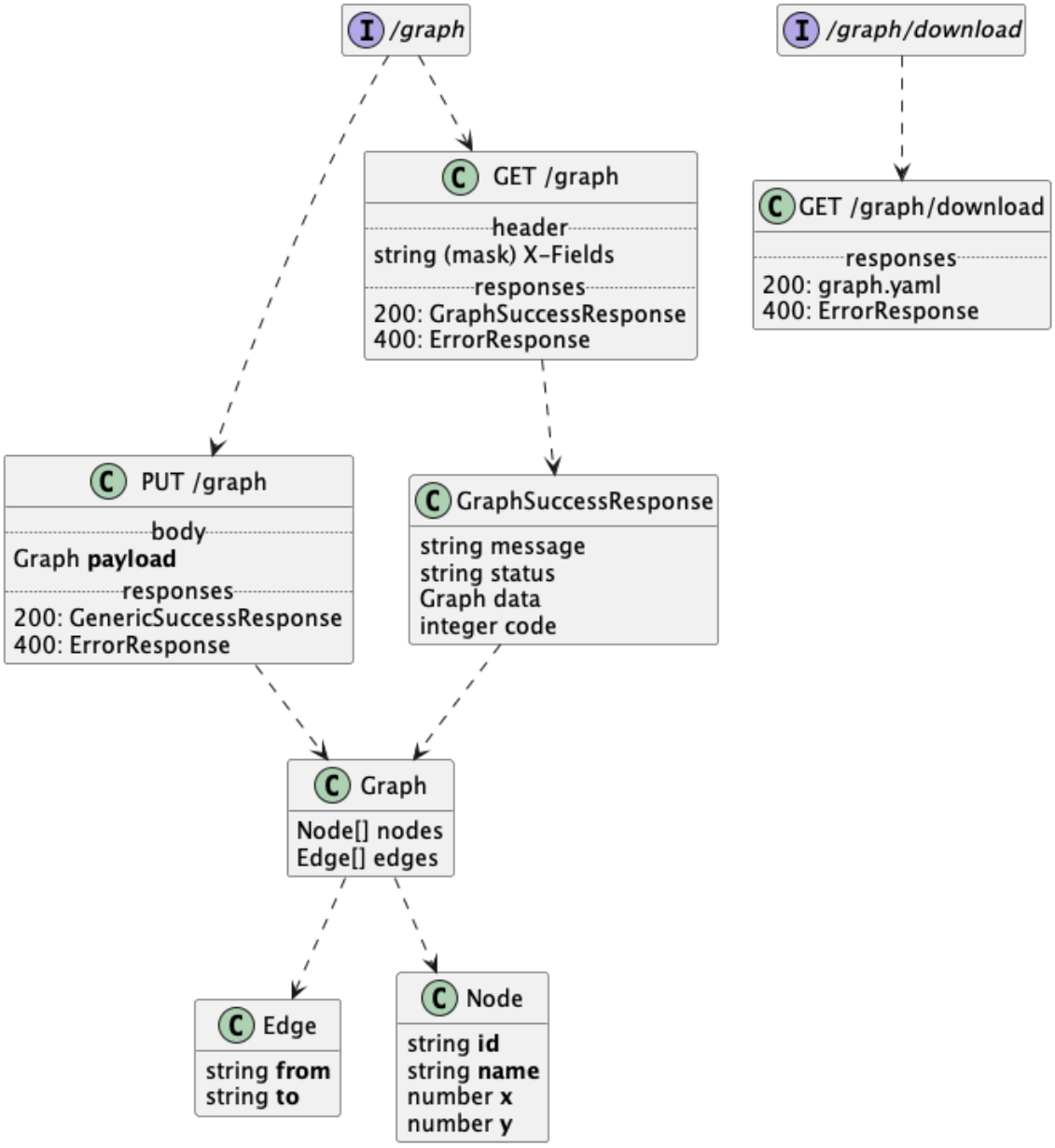
```python
def write_csv_data_to_file():
    with open(csv_file_path, "a", newline="") as csvfile:
        fieldnames = ["identity", "datetime", "major", "minor", "tx_power", "rssi"]
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
        for data in csv_data_cache:
            writer.writerow(data)
    csv_data_cache.clear()
```

Figure 1 - Écriture des données dans un fichier .csv

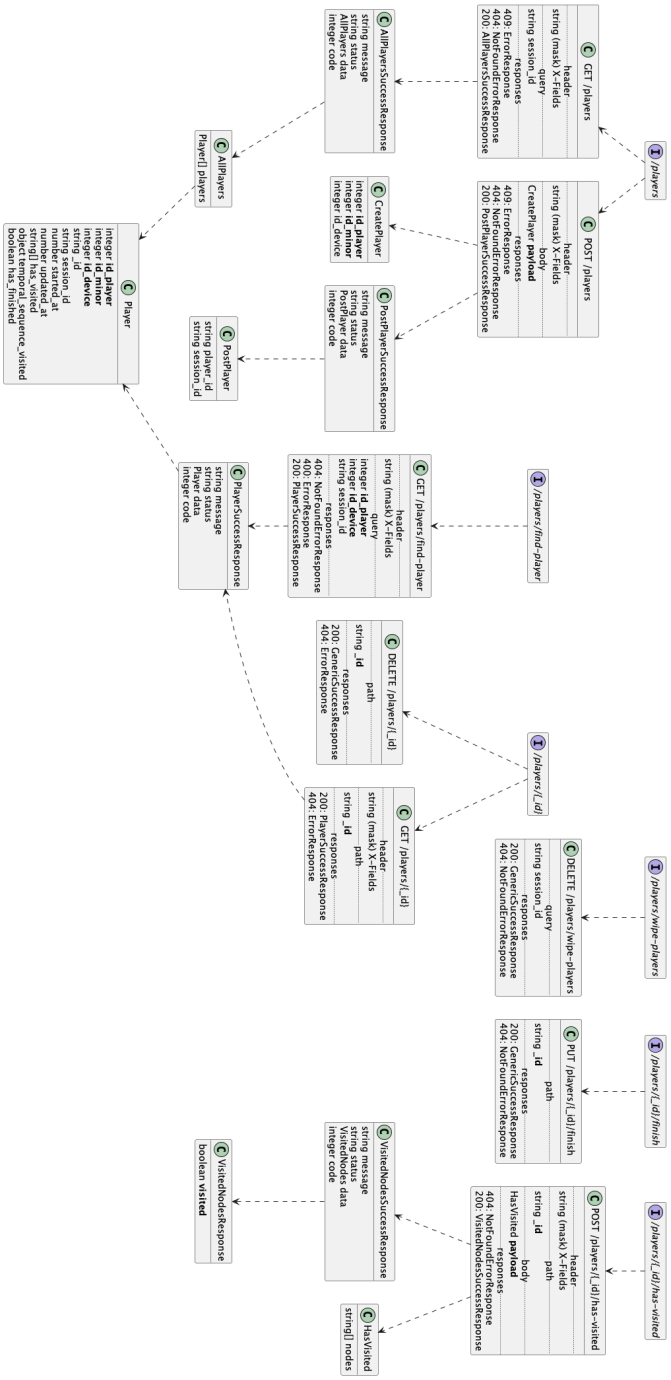
15.3 API pour la gestion de l'inventaire



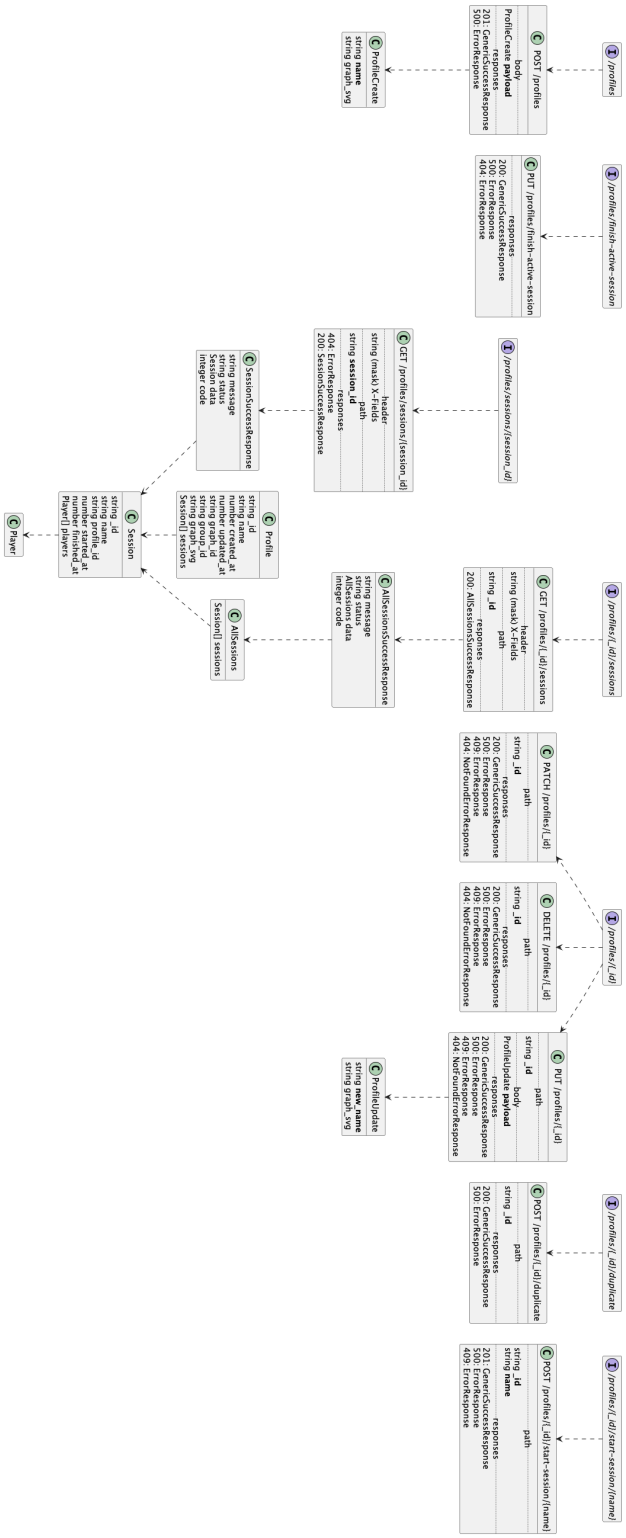
15.4 API pour la gestion du graphe



15.5 API pour la gestion des joueurs



15.6 API pour la gestion des profils



15.7 Documentation des collections MongoDB

1 Profiles

Attribut	Description	Type
_id	Identifiant unique du profil	ObjectID
name	Nom du profil	String
created_at	Timestamp de création	Double (timestamp)
updated_at	Timestamp de la dernière mise à jour	Double (timestamp)
graph_id	Identifiant du graphique associé	ObjectID
group_id	Identifiant du groupe associé	ObjectID
graph_svg	Nom du fichier SVG de la carte	String
sessions	Liste des identifiants de sessions associées	List[ObjectID]

Table 1: Profiles Summary

```
{
    _id: ObjectId('6658920eb5dcf0a3a1deb016'),
    name: 'FDSL 2023',
    created_at: 1717080590.774672,
    updated_at: 1717751658.321927,
    graph_id: ObjectId('6658920eb5dcf0a3a1deb014'),
    group_id: ObjectId('6658920eb5dcf0a3a1deb015'),
    graph_svg: 'la-rochelle-new.svg',
    sessions: [
        ObjectId('665d76ff2d662df6e5ff7158'),
        ObjectId('665dab58860da615211d6722'),
        ObjectId('6662ee16ce9693f2192be93c'),
        ObjectId('6663183e2b29d5f3df55a98a')
    ]
}
```

2 Sessions

Attribut	Description	Type
_id	Identifiant unique de la session	ObjectID
profile_id	Identifiant du profil associé	ObjectID
name	Nom de la session	String
started_at	Timestamp du début de la session	Double (timestamp)
finished_at	Timestamp de fin de la session	Double (timestamp)
players	Liste des identifiants des joueurs associés	List[ObjectID]

Table 2: Sessions Summary

```
{
    _id: ObjectId('6663183e2b29d5f3df55a98a'),
```

```
    profile_id: ObjectId('6658920eb5dcf0a3a1deb016'),
    name: 'FDSL 2023 - Matin',
    started_at: 1717770302.432315,
    finished_at: 1717770345.464234,
    players: [
        ObjectId('666318592b29d5f3df55a98b')
    ]
}
```

3 Groups

Attribut	Description	Type
_id	Identifiant unique du groupe	ObjectID
hosts	Liste des identifiants des hôtes associés	List[ObjectID]

Table 3: Groups Summary

```
{
    _id: ObjectId('6658920eb5dcf0a3a1deb015'),
    hosts: [
        ObjectId('665d87bb2d662df6e5ff715a'),
        ObjectId('665da5df860da615211d6719'),
        ...
    ]
}
```

4 Hosts

Attribut	Description	Type
_id	Identifiant unique de l'hôte	ObjectID
name	Nom de l'hôte	String
active	Statut de présence dans l'inventaire Ansible	Boolean
node	Identifiant du noeud associé	String

Table 4: Hosts Summary

```
{
    _id: ObjectId('665d87bb2d662df6e5ff715a'),
    name: 'rasp13.local',
    active: false,
    node: '2'
}
```

5 Players

Attribut	Description	Type

_id	Identifiant unique du joueur	ObjectID
id_player	Identifiant du joueur	Integer
id_minor	Identifiant "minor" de l'émetteur BLE associé au joueur	Integer
id_device	Identifiant de l'appareil de "jeu"	Integer
session_id	Identifiant de la session associée	ObjectID
started_at	Timestamp du début de la participation du joueur	Double (timestamp)
updated_at	Timestamp de la dernière mise à jour	Double (timestamp)
has_visited	Liste des noeuds visités	List
temporal_sequence_visited	Séquence temporelle des noeuds visités	Dict
has_finished	Statut d'achèvement de la session	Boolean

Table 5: Players Summary

```
{
    _id: ObjectId('665d770d2d662df6e5ff7159'),
    id_player: 1,
    id_minor: 1,
    id_device: 1,
    session_id: ObjectId('665d76ff2d662df6e5ff7158'),
    started_at: 1632040864,
    updated_at: 1632040864,
    has_visited: [],
    temporal_sequence_visited: {
        '1632040864.0': {
            from: null,
            to: [
                '34'
            ],
            score: 1.298513943015059
        },
        '1632041222.0': {
            from: '34',
            to: [
                '12',
                '17',
                '18'
            ],
            score: 0.7031546336335737
        },
        ...
    },
    has_finished: false
}
```

6 Graphs

Attribut	Description	Type
_id	Identifiant unique du graphique	ObjectID
nodes	Liste des noeuds, chacun avec un identifiant, nom, x, y	List[Dict]
edges	Liste des arêtes, chacune avec un noeud de départ et de fin	List[Dict]

Table 6: Graphs Summary

```
{
    _id: ObjectId('6658920eb5dcf0a3a1deb014'),
    nodes: [
        {
            id: '2',
            name: 'Noeud',
            x: 458.9201520912548,
            y: 194.61216730038024
        },
        {
            id: '12',
            name: 'Noeud',
            x: 523.2877767083735,
            y: 127.56496631376324
        },
        {
            id: '34',
            name: 'Noeud',
            x: 576.8669201520913,
            y: 86.31558935361217
        }
    ],
    edges: [
        {
            from: '2',
            to: '12'
        },
        {
            from: '12',
            to: '34'
        }
    ]
}
```

7 Cache

Attribut	Description	Type
_id	Identifiant unique de l'entrée de cache	ObjectID
key	Clé de l'entrée de cache	String
value	Valeur de l'entrée de cache (variable selon la clé)	Variable

Table 7: Cache Summary

```
{
    _id: ObjectId('6658920e07608d769cf7f358'),
    key: 'active_profile',
    value: ObjectId('6658920eb5dcf0a3a1deb016')
}
```